MSH 3-T-junction array

Function reference

`msh_tjunctions_tn.ipynb`	Julia, TensorBinding.jl / ITensors: MPO, DMRG, TDVP
`tjunctions_python_ed.ipynb`	Python, kwant: exact diagonalization
`tjunctions_compare.ipynb`	Python: comparison of the two

Repository `MSH_Tensor_Network_Methods`

29 October 2026

This document explains every function and struct defined in the three notebooks: what it computes, the method, its inputs and outputs, the numerical settings that matter, and where it is used. All numbers are quoted from the saved outputs of the executed notebooks. Function names in `typewriter` are those of the code.

Contents

1 Overview **3**

1.1 Model and parameters . . . 3
1.2 Geometry . . . 3
1.3 Encoding of the lattice in the MPO . . . 4
1.4 Nambu bases of the two codes . . . 4
1.5 Logical qubit and readout . . . 4
1.6 Workflow and data files . . . 5
1.7 Mapping between the two codes . . . 6

2 Part I — Tensor-network notebook (`msh_tjunctions_tn.ipynb`) **7**

2.1 Setup (cell 1) . . . 7
2.2 Section 1: Hamiltonian, MPO and dense reference . . . 7
2.3 Section 2: protocols for $\sqrt{Z}$, $\sqrt{X}$ and the partial fusion . . . 9
2.4 Section 3a: time evolution and readout . . . 10
2.5 Section 3b: zero modes from DMRG, refinement and localization . . . 12
2.6 Section 4: check of the MPO and export of `tb_reference/` . . . 13
2.7 Section 5: subgap LDOS and MZMs from DMRG . . . 14
2.8 Section 6: gates followed by partial fusion with TDVP . . . 14
2.9 Section 7: ΔQ during the fusion and bond dimension . . . 16
2.10 Section 8: summary . . . 17
2.11 Section 9: cost of the tensor-network route . . . 17

3 Part II — Python/kwant notebook (`tjunctions_python_ed.ipynb`) **19**

3.1 Setup (cell 1) . . . 19
3.2 Section 1: geometry . . . 19

3.3 Section 2: spin texture and protocols . . . 19
3.4 Section 3: Hamiltonian . . . 20
3.5 Section 4: check against the Julia/TensorBinding array . . . 20
3.6 Section 5: subgap LDOS . . . 20
3.7 Section 6: Majoranas, fermion modes and logical states . . . 21
3.8 Sections 7–8: Z and X gates, then the partial fusion . . . 24
3.9 Section 9: charge density $\delta n(x,t)$ . . . 25
3.10 Section 10: saving the results . . . 25
3.11 Section 11: cost of the dense route . . . 25

4 Part III — Comparison notebook (`tjunctions_compare.ipynb`) 27
4.1 Helper functions . . . 27
4.2 Section 1: subgap LDOS (row 1) . . . 27
4.3 Section 2: TDVP $\delta n(x,t)$ and $\Delta Q(t)$ (rows 2–5) . . . 27
4.4 Section 3: leakage and fusion quantities (rows 6–17) . . . 28
4.5 Results table . . . 29
4.6 Section 4: cost and scaling . . . 29
4.7 Section 5: summary . . . 30

1 Overview

1.1 Model and parameters

Both codes describe a magnet–superconductor hybrid (MSH) array of chain sites on a square lattice, without substrate. The Bogoliubov–de Gennes (BdG) Hamiltonian in the Nambu basis $(c_\uparrow, c_\downarrow, c_\uparrow^\dagger, c_\downarrow^\dagger)$ of the tensor-network (TN) notebook is

$$H_{\mathrm{BdG}} = \begin{pmatrix} H_0 & \Delta\, i\sigma_y \\ (\Delta\, i\sigma_y)^\dagger & -H_0^* \end{pmatrix}, \qquad H_0 = \sum_{\mathbf{r}} c_{\mathbf{r}}^\dagger(-\mu + J\,\mathbf{S}_{\mathbf{r}}\cdot\sigma)c_{\mathbf{r}} + \sum_{\mathbf{r},\hat{d}} c_{\mathbf{r}+\hat{d}}^\dagger(-t + i\alpha(d_x\sigma_y - d_y\sigma_x))c_{\mathbf{r}},$$

where $\hat{d}$ runs over unit bonds between existing sites. The parameters are the same in both codes:

Symbol	Value	Code name (Julia / Python)	Meaning
t	1	`t` / `t`	nearest-neighbour hopping
μ	-7	`μ` / `mu`	chemical potential
α	0.9	`α` / `alpha`	Rashba spin–orbit coupling
Δ	2.4	`Δ` / `Delta`	s-wave pairing
J	5.8	`J` / `J`	exchange coupling to the magnetic moments

Spin texture. Each site carries a classical moment $\mathbf{S} = (\cos\phi\sin\theta,\ \sin\phi\sin\theta,\ \cos\theta)$ with

$$\theta = (-1)^{x+y}\,\frac{\pi}{2}f, \qquad \phi = \theta_{\mathrm{arm}} + \frac{\pi}{2},$$

where $\theta_{\mathrm{arm}} = 0, \pi, \pi/2$ for the right, left and upward arm. The rotation fraction $f \in [0,1]$ is set per site: $f = 0$ gives $\mathbf{S} \parallel \hat{z}$ (topological), $f = 1$ an in-plane antiferromagnet perpendicular to the arm (trivial). All gates and the fusion are written as time-dependent textures $f(c, s, n; t)$.

1.2 Geometry

The array has three T-junctions (cells $c = 1, 2, 3$) with junction sites at $x_c = -32, 0, 32$ on the horizontal chain $y = 0$, $x = -48 \ldots 47$. Each cell consists of three arms of $2^L = 16$ sites:

Arm s	Direction	First site ($n = 0$)	Sites
1	right $(+\hat{x})$	junction site $(x_c, 0)$	$x = x_c \ldots x_c + 15$
2	left $(-\hat{x})$	$(x_c - 1, 0)$	$x = x_c - 1 \ldots x_c - 16$
3	up $(+\hat{y})$	$(x_c, 1)$	$y = 1 \ldots 16$

A site is labelled (c, s, n) with $n = 0 \ldots 15$ along the arm; the position is $\mathbf{r}(c, s, n) = (x_c, 0) + \mathbf{o}_s + n\,\hat{e}_s$ with origins $\mathbf{o}_s = (0,0), (-1,0), (0,1)$ and arm directions $\hat{e}_s$ (`array_pos` in both codes). The array has $3\times3\times16 = 144$ sites and the BdG dimension is $D = 4\times144 = 576$. Neighbouring cells are joined end to end: the right end of cell c $(x_c + 15)$ bonds to the left end of cell $c + 1$ $(x_{c+1} - 16 = x_c + 16)$.

Initial setup. The texture `f0` sets $f = 1$ (trivial) on cell 2 and on every vertical leg, and $f = 0$ (topological) on the horizontal legs of cells 1 and 3. The topological segments $x = -48 \cdots -17$ and $x = 16 \ldots 47$ each host two Majorana zero modes (MZMs): γ_1, γ_2 at the ends of cell 1's segment and γ_3, γ_4 at cell 3's. Their position expectation values are $-46.5, -17.5, 16.5, 45.5$ in the TN code (coordinate $\mathbf{r}\cdot\hat{u}$ with $\hat{u}$ at angle 0.05) and $x = -46.6, -17.5, 16.5, 45.6$ in the Python code. The fully trivial junction 2 has an Andreev bound state at $E = \pm 0.127$; the next state is at $|E| = 0.268$.

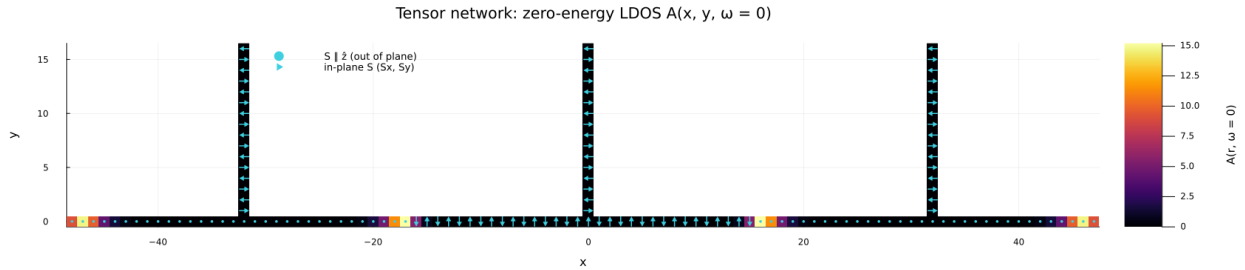


Figure 1: Zero-energy LDOS $A(x, y, \omega = 0)$ of the initial setup from DMRG (`msh_tjunctions_tn.ipynb`, section 5), with the initial spin texture: dots are moments along $\hat{z}$ (topological segments), arrows the in-plane moments of the trivial regions (cell 2 and all vertical legs). The four MZMs sit at the ends of the two topological segments.

1.3 Encoding of the lattice in the MPO

The TN notebook stores a single-particle BdG vector as an MPS (and H as an MPO) on eight sites

$$[\,\text{nambu};\ \text{spin};\ \text{cell};\ \text{pos}_1, \text{pos}_2, \text{pos}_3, \text{pos}_4;\ \text{arm}\,]$$

with dimensions $2, 2, 3, 2, 2, 2, 2, 3$ (product $576 = D$). The position n along an arm is written in binary ("quantics") on the four qubits, with pos_1 the most significant bit:

$$n = \sum_{k=1}^{4} b_k\, 2^{4-k}, \qquad b_k \in \{0, 1\}.$$

An operator diagonal in position, $\sum_n v_n |n\rangle\langle n|$, is an MPO on the qubits (`diag_mpo_exact`); hopping along an arm is the binary-adder shift $|n\rangle \to |n+1\rangle$ (an MPO of bond dimension 2 from TensorBinding, non-cyclic, so $n = 15$ is not joined to $n = 0$); bonds at the junction and between cells use the rank-1 projectors $|0\rangle\langle 0|$ and $|15\rangle\langle 15|$ on the qubits combined with $|b\rangle\langle a|$ operators on the arm and cell indices. The full H has bond dimension $\chi(H) = 18$; with the moving exchange of a protocol it is 21–22. The dense form of the MPO (`dense`) uses the order nambu ⊗ spin ⊗ cell ⊗ pos ⊗ arm, arm fastest.

1.4 Nambu bases of the two codes

	TN (Julia)	Python (kwant)
Basis per site	$(c_\uparrow, c_\downarrow, c_\uparrow^\dagger, c_\downarrow^\dagger)$	$(c_\uparrow, c_\downarrow, c_\downarrow^\dagger, -c_\uparrow^\dagger)$
Component order	nambu ⊗ spin ⊗ site	site-major, 4 site + orbital
On-site term	$-\mu + J\mathbf{S}\cdot\sigma$; pairing $\Delta\, i\sigma_y$	$-\mu\tau_z + \Delta\tau_x + J\,\tau_0\mathbf{S}\cdot\sigma$
Hopping $(+\hat{x}, +\hat{y})$	$-t + i\alpha\sigma_y,\ -t - i\alpha\sigma_x$	$-t\tau_z + i\alpha\tau_z\sigma_y,\ -t\tau_z - i\alpha\tau_z\sigma_x$
Particle–hole operator	$C = \tau_x K$	$C = \tau_y\sigma_y K$
Charge operator	τ_z on the Nambu site	`tauz` $= (1, 1, -1, -1)$ per site

Per site, the TN components are W times the kwant components: $(u_\uparrow, u_\downarrow, v_\uparrow, v_\downarrow) = (a, b, -d, c)$ for kwant components $(a, b, c, d) = (u_\uparrow, u_\downarrow, v_\downarrow, -v_\uparrow)$. The Python notebook builds this map as a permutation-with-sign matrix (`to_tb_basis`) and finds $\|H_{\text{kwant}} - H_{\text{TensorBinding}}\| = 3.5 \times 10^{-12}$ for the initial texture and $\le 2 \times 10^{-9}$ at three protocol points.

1.5 Logical qubit and readout

The two zero-mode fermions are

$$\psi_A = (\gamma_1 + i\gamma_2)/\sqrt{2}, \qquad \psi_B = (\gamma_3 + i\gamma_4)/\sqrt{2},$$

taken as the positive-energy members of each $\pm E$ pair. The logical qubit is the even-parity pair $|0\rangle_L = |00\rangle$, $|1\rangle_L = |11\rangle$ (occupations of ψ_A, ψ_B). Both states share the same filled negative-energy bulk; they differ only in the occupation of the two fermions. In single-particle language, $|11\rangle$ occupies ψ_A, ψ_B while $|00\rangle$ occupies their particle–hole partners $C\psi_A, C\psi_B$. The electron density of an occupied state ϕ is $|u_\phi|^2$, and $|u_{C\psi}|^2 = |v_\psi|^2$. The bulk therefore cancels in the difference and

$$\delta n(\mathbf{r}, t) = n_{\hat{U}|11\rangle}(\mathbf{r}) - n_{\hat{U}|00\rangle}(\mathbf{r}) = \sum_{k=A,B} \sum_{\sigma} (|u_{k\sigma}(\mathbf{r}, t)|^2 - |v_{k\sigma}(\mathbf{r}, t)|^2), \qquad \Delta Q = \sum_{\mathbf{r}} \delta n = \sum_k \langle \psi_k | \tau_z | \psi_k \rangle.$$

ΔQ is an electron number; the charge is $-e\,\Delta Q$. The TN code evaluates the right-hand side from the two evolved MPS only. The Python code evolves the full $4N \times 4N$ basis and evaluates the left-hand side from $G = WW^\dagger$ of the occupied columns; the two are equal by the argument above.

Leakage. The leakage of mode k out of the initial zero-mode subspace is

$$\ell_k = 1 - \sum_{i=1}^{4} |z_{ik}|^2, \qquad z_{ik} = \langle \gamma_i(0) | \psi_k(t) \rangle.$$

At the end of the partial fusion, the leakage is measured against the four lowest states of the final Hamiltonian instead.

Gates. `exchange_12` exchanges γ_1 and γ_2 around junction 1's vertical leg; applied once it is $\sqrt{Z}$, twice Z. `exchange_23` exchanges γ_2 and γ_3 through the trivial junction 2; once is $\sqrt{X}$, twice X. The partial fusion of pair A (cell 1) makes the cell trivial except $2L_p + 1 = 3$ sites around the junction, so that ψ_A acquires a finite energy ($E_A = 0.0439$ at the end) and the two logical states acquire different charges. $\Delta Q = Q(\hat{U}|1\rangle) - Q(\hat{U}|0\rangle)$ compares the two evolved logical states: after Z or no gate each logical state is mapped to itself and ΔQ is close to the adiabatic value -0.4769 of pair A (-0.4777 and -0.4774); after X the two are interchanged and the sign flips ($+0.4770$).

1.6 Workflow and data files

Run order (all in the repository directory):

`msh_tjunctions_tn.ipynb` → `tjunctions_python_ed.ipynb` → `tjunctions_compare.ipynb`.

The Python notebook needs `tb_reference/` from the TN notebook; the comparison notebook only loads files.

File	Content (written by)
`tb_reference/coords.txt`, `*.bin`	site coordinates and dense MPO matrices at four protocol points (TN, sec. 4)
`msh_array_ldos_tn.csv`	DMRG LDOS $A(x, y, \omega)$ (TN, sec. 5)
`msh_tjunctions_tdvp_{Z,X,F}.jls`	cached TDVP results, both modes (TN, sec. 6)
`tdvp_progress.log`, `tdvp_status.txt`	TDVP progress log and status file (TN, sec. 6)
`msh_gate_density_tdvp_{Z,X}.csv`, `msh_gate_dQ_tdvp_{Z,X}.csv`	$\delta n(x, t)$ on the horizontal chain and $\Delta Q(t)$ (TN, sec. 6)
`msh_tjunctions_tn_gates.csv`	12 summary numbers, `key,value` (TN, sec. 8)
`msh_tjunctions_tn_timing.csv`	wall times and scaling benchmark (TN, sec. 9)
`.../array_ldos_kwant.npz`	dense LDOS (Python, sec. 5); `...` = `tjunctions_python_ed_data/`
`.../gate_density_kwant_{Z,X400}.npz`	$\delta n(x, t)$, $\Delta Q(t)$ (Python, sec. 9)
`.../gates_kwant.npz`	the 12 summary numbers under the same keys (Python, sec. 10)
`.../timing_kwant.npz`	wall times and scaling benchmark (Python, sec. 11)

Runtimes stated in the notebooks: the TN notebook takes about 10 minutes with the TDVP cache and about 2.5 hours for a first TDVP run with six Julia threads (`JULIA_NUM_THREADS=6`); the Python notebook about 35 minutes; the comparison seconds.

1.7 Mapping between the two codes

The compare rows are numbered in the order they are printed by `tjunctions_compare.ipynb` (Table 1).

Quantity	TN (`msh_tjunctions_tn`)	Python (`tjunctions_python_ed`)	Compare row(s)
Geometry	`TArray`, `array_pos`, `center_x`	`label`, `array_pos`, `center_x`	–
Texture	`array_spin`	`spins`	–
Protocols	`Exchange12`, `Exchange23`, `Fusion`, `f_array_braid`, `f_array_23`, `f_fuse`	same names (`f_array_12`)	–
H	`msh_tjunction_array` (MPO)	`build_system_3T`, `ham_kwant`	TN sec. 4, Py sec. 4
$H(t)$	`MovingArrayH`	`ham`, `ham_sp`	–
Subgap LDOS	`zero_mode_subspace`, `ritz`, `site_uv`, `gauss`	`la.eigh`, `w_site`, `g_eta`	1
Majoranas	`localize_majoranas`, `clean_majoranas`	`individual_MZMs` (`takagi`, `PHconj`)	–
ψ_A, ψ_B	`pair_modes`	`fix_unitary`, `masks`	–
Time evolution	`evolve_mode_tdvp` (TDVP, $dt = 0.1$)	`evolve`, `step` (Gauss–Legendre, $dt = 0.25$)	–
$\delta n(x,t)$	`site_dn`	`electron_density`, `readout`	2, 4
$\Delta Q(t)$	`ΔQ_mps`	`readout` (`dQ`)	3, 5
Leakage after gates	`mode_leak`(z)	`mode_leak`(R)	6–9
Fusion ΔQ	`analyze` (`:fusion`)	`run` (`"fusion"`)	10–12
Fusion leakage	`fusion_leak`	`lk` in `run`	13–15
Adiabatic ΔQ, E_A	`ΔQ_mps(φF[iA])`, `EF[iA]`	`U_end[:, iA]`, `E_end[iA]`; `dQ_adiabatic`	16, 17
Cost / scaling	`bench_tdvp`, section 9	`ham_nleg`, `bench_step`, section 11	cost section

2 Part I — Tensor-network notebook (`msh_tjunctions_tn.ipynb`)

Every reported quantity in this notebook comes from MPO/MPS methods: DMRG for the subgap states and the Majoranas, TDVP for the gates and the fusion readout. Dense matrices appear only in section 4, to check that the MPO equals the lattice Hamiltonian. Packages: `ITensors`, `ITensorMPS`, `TensorBinding` (alias `TB`), `Plots`, `Serialization`.

2.1 Setup (cell 1)

Global constants: `ROOT = pwd()` (all files are read and written in the notebook's folder); `RUN_TDVP = false` (set to `true` to rerun the TDVP of section 6 even when the cache exists); the Pauli matrices `σ0, σx, σy, σz`.

`quiet(f)`

```
quiet(f) = with_logger(f, ConsoleLogger(stderr, Logging.Error))
```

Runs the closure `f` with a logger that shows errors only. KrylovKit warns at every non-converged local solve in DMRG on K and in `linsolve`; the notebook comment calls these warnings harmless. Used around `zero_mode_subspace` and `refine_subspace`.

2.2 Section 1: Hamiltonian, MPO and dense reference

`diag_mpo_exact(v, pos; cutoff=1e-14)`

```
function diag_mpo_exact(v::AbstractVector, pos; cutoff=1e-14)
    A = permutedims(reshape(ComplexF64.(v), ntuple(_ -> 2, L)), L:-1:1)
    ψ = MPS(A, pos; cutoff=cutoff)
    ...  δ[s => j, s' => j, s'' => j] = 1
    W[k] = replaceprime(ψ[k] * δ, 2 => 0; tags=tags(s))
```

Builds the exact MPO of the diagonal operator $\sum_n v_n |n\rangle\langle n|$ on the L position qubits (pos_1 = most significant bit). *Method:* the length-2^L vector is reshaped into an L-index tensor (the `permutedims` puts the most significant bit first), compressed into an MPS by TT-SVD with `cutoff = 1e-14`, and each MPS tensor is contracted with a three-leg Kronecker delta $\delta_{ss's''}$ so that the physical index becomes a diagonal operator index pair. The result is exact up to the cutoff. *Used in:* the site-dependent exchange field (`msh_tjunction_array`, `array_exchange_mpo`) and the position operator (`array_position_mpo`).

`dense(mpo, sites)`

Contracts the whole MPO (`prod`) and reshapes it into a $D \times D$ matrix, big-endian over `sites` (first site most significant). Only used for the checks of section 4 and the `tb_reference/` export.

`hop_matrix(t, α, d)`

```
hop_matrix(t, α, d) = -t * σ0 + im * α * (d[1] * σy - d[2] * σx)
```

The 2×2 spin matrix of the hopping $c^\dagger_{\mathbf{r}+\hat{d}} h\, c_{\mathbf{r}}$ along the unit bond $\hat{d} = (d_x, d_y)$: $h = -t + i\alpha(d_x\sigma_y - d_y\sigma_x)$. The reverse bond has $h(-\hat{d}) = h(\hat{d})^\dagger$.

`struct TArray; Ncell, center_x, array_pos, array_spin, ARM_ANG, ARM_ORIG`

```
Base.@kwdef struct TArray
    L::Int = 4          # 2^L = 16 sites per arm
    ncell::Int = 3
end
Ncell(a::TArray) = 2^a.L
center_x(a::TArray, c) = (c - (a.ncell + 1) / 2) * 2Ncell(a)
array_pos(a, c, s, n) = (center_x(a, c), 0.0) .+ ARM_ORIG[s] .+ n .* (cos(ARM_ANG[s]),
 ↪  sin(ARM_ANG[s]))
```

`TArray` holds the geometry: L position qubits (2^L sites per arm) and the number of cells. `Ncell` returns 2^L (sites per arm, despite the name). `center_x` gives the junction coordinate $x_c = (c - (n_{\text{cell}}+1)/2)\cdot 2^{L+1}$, i.e. $-32, 0, 32$ for $L = 4$. `array_pos` returns $\mathbf{r}(c,s,n)$ with `ARM_ANG = (0, π, π/2)` and `ARM_ORIG = ((0,0), (-1,0), (0,1))`. `array_spin(a, f)` returns a closure $(c,s,n) \mapsto \mathbf{S}$ with $\theta = (-1)^{x+y}(\pi/2)f(c,s,n)$ and $\phi = \theta_{\text{arm}} + \pi/2$, using the rounded integer coordinates for the sign.

`msh_tjunction_array(a, f; t, μ, α, Δ, J, cutoff=1e-13, sites=nothing)`

```
function msh_tjunction_array(a::TArray, f; t=1.0, μ=-7.0, α=0.9, Δ=2.4, J=5.8, cutoff=1e-13,
 ↪  sites=nothing)
    ...
    return HB, [ns; ss; cl; pos; arm]
end
```

Builds the BdG Hamiltonian of the array with texture `f` as an MPO and returns it with the site list `[nambu; spin; cell; pos₁…pos₄; arm]`. If `sites` is given, the existing indices are reused, so MPS built on one Hamiltonian can be used with another (needed for $H(t)$).

Construction. A helper `term(Mpos, σmat, Bcell, Barm)` takes an MPO on the position qubits and attaches an operator on the arm (`TB.postpend_op`), the cell and the spin (`TB.prepend_op`). `P(d,i)` is the projector $|i\rangle\langle i|$ and `E(d,i,j)` $= |i\rangle\langle j|$. The normal part H_0 is the sum of

1. the chemical potential $-\mu \cdot \mathbb{1}$;
2. intra-arm hopping for each arm s: $K_+ \otimes h(\hat{e}_s) \otimes \mathbb{1}_{\text{cell}} \otimes P_s$ and $K_- \otimes h(\hat{e}_s)^\dagger \otimes \mathbb{1} \otimes P_s$, where (K_+, K_-) = `TB.shift_pair_mpos(pos, 1; cyclic=false)` are the binary shift $|n\rangle \to |n+1\rangle$ and its adjoint;
3. junction bonds through the site-0 projector `P0 = TB._site_projector_mpo(L, pos, 0)`, with arm operators $|b\rangle\langle a|$ for the four directed bonds $(a \to b, \hat{d})$: $(2 \to 1, +\hat{x})$, $(1 \to 2, -\hat{x})$, $(3 \to 1, -\hat{y})$, $(1 \to 3, +\hat{y})$. These connect the junction site (arm 1, $n = 0$) with its left neighbour (arm 2, $n = 0$) and with the first site of the vertical leg (arm 3, $n = 0$);
4. inter-cell bonds through the site-15 projector `Pend`, with cell operator $|c+1\rangle\langle c|$ and arm operator $|2\rangle\langle 1|$ (right end of cell c to the far end of cell $c+1$'s left arm) and the Hermitian conjugate;
5. the exchange $J\,\mathbf{S}\cdot\sigma$: for each cell, arm and component k the vector $v_n = JS_k(c,s,n)$ becomes a diagonal MPO (`diag_mpo_exact`), with σ_k on spin and projectors on cell and arm; zero components are skipped.

The terms are summed with `cutoff = 1e-13`. The BdG MPO is then

$$H_B = \tau_{pp} \otimes H_0 + \tau_{hh} \otimes (-H_0^*) + \tau_+ \otimes (\Delta\, i\sigma_y) + \tau_- \otimes (\Delta\, i\sigma_y)^\dagger,$$

using `TB.prepend_nambu` with `:Pp`, `:Ph`, `:tp` ($|p\rangle\langle h|$), `:tm` ($|h\rangle\langle p|$); $-H_0^*$ is `-1.0 * dag(H0)` and the pairing is `ComplexF64[0 1; -1 0]` $= i\sigma_y$. The resulting MPO has $\chi = 18$. *Used in:* section 4 (`HB`), as the $J = 0$ base of `MovingArrayH`, and in `bench_tdvp`.

`array_reference(a, f; t, μ, α, Δ, J)`

Independent dense BdG matrix (576×576) built only from site coordinates, in the same basis as `dense(HB, sites)`. The index of site (c, s, n) is `idx = (c-1)*3N + 3n + s` (cell ⊗ pos ⊗ arm, arm fastest); spin blocks are `[i, M+i]`. On-site blocks $-\mu + J\mathbf{S}\cdot\sigma$; for every ordered pair of sites exactly one lattice spacing apart (`hypot(d) ≈ 1`), $H_0[\mathbf{r} + \hat{d}, \mathbf{r}] \mathrel{+}= h(\hat{d})$. This brute-force double loop does not use the arm/junction/cell logic of the MPO, which makes it an independent check. Pairing block $\Delta\, i\sigma_y \otimes \mathbb{1}_M$; result $\left(\begin{smallmatrix} H_0 & D \\ D^\dagger & -H_0^* \end{smallmatrix}\right)$.

`array_exchange_mpo(a, sites, f, cells; J=5.8, cutoff=1e-14)`

The exchange of the listed `cells` only, as a BdG MPO on `sites`: particle part $\tau_{pp} \otimes \sigma_k$, hole part $\tau_{hh} \otimes (-\sigma_k^*)$, each times the diagonal MPO of JS_k and cell/arm projectors. Returns `nothing` if all components vanish. This is what makes $H(t)$ cheap: only the exchange of moving cells is rebuilt at each time (`MovingArrayH`).

`array_position_mpo(a, sites; β=0.05)`

Diagonal position operator $\hat{X} = \sum_{\mathbf{r}} (\mathbf{r}\cdot\hat{u})|\mathbf{r}\rangle\langle\mathbf{r}| \otimes \mathbb{1}_{\text{spin}} \otimes \mathbb{1}_{\text{nambu}}$ with $\hat{u} = (\cos\beta, \sin\beta)$, $\beta = 0.05$. Built per cell and arm from `diag_mpo_exact`. Used by `localize_majoranas`; the eigenvalues it produces are the "centres $\mathbf{r}\cdot\hat{u}$" printed in section 5.

`site_list(a)`

All sites as (c, s, n), ordered cell, then n, then arm. This is the row order of the LDOS CSV and of `coords.txt`.

2.3 Section 2: protocols for $\sqrt{Z}$, $\sqrt{X}$ and the partial fusion

All protocols are textures $f(c, s, n; t)$ built from

$$\mathrm{ss}(\tau) = \begin{cases} 0 & \tau \le 0 \\ \sin^2(\pi\tau/2) & 0 < \tau < 1 \\ 1 & \tau \ge 1 \end{cases} \qquad \texttt{(smoothstep)},$$

with $\tau = t/T_{\text{rot}}$ and a site-dependent delay `step_T`$\cdot d$: each site turns from topological to trivial (or back) over one `rot_T`, starting at a delay that depends on its position, so the topological/trivial domain wall moves along the leg. The ramps were written for legs of $L_1 = L_2 = 14$ sites (a 1D index i along the horizontal segment and k along the vertical leg) and are mapped onto the 16-site legs of the array.

`smoothstep(τ), f0`

`smoothstep` as above. `f0(c,s,n)` $= 1$ if $c = 2$ or $s = 3$, else 0: the initial setup with four MZMs.

`struct Exchange12; duration(p::Exchange12)`

```
Base.@kwdef struct Exchange12
    L1::Int = 14; L2::Int = 14; rot_T::Float64 = 200.0; step_T::Float64 = 0.1
end
duration(p::Exchange12) = (p.step_T * (4p.L1 + 2p.L2 + 8) + 1) * p.rot_T
```

Parameters of `exchange_12` (`rot_Tz = 200`, `step_Tz = 0.1`). Duration of one exchange $T_{12} = (0.1 \cdot 92 + 1) \cdot 200 = 2040$; the Z gate is two exchanges, $T_{\text{gate}} = 4080$.

`f_horizontal(p, i, t), f_vertical(p, k, t)`

`f_horizontal` is the rotation fraction of horizontal site i of the original L_1-geometry: $i = 0 \dots L_1$ is the left half, $i = L_1 + 1$ the junction (always 0, topological), $i = L_1 + 2 \dots 2L_1 + 2$ the right half; elsewhere 1. Each half is a difference of two delayed smoothsteps, i.e. the site becomes trivial at one delay and topological again at a later one. `f_vertical` is $1 - \mathrm{ss}(\tau - s(k + L_1 + 2)) + \mathrm{ss}(\tau - s(L_2 - k + 3L_1 + L_2 + 5))$ for vertical site k: the leg becomes topological from the junction outward and trivial again from the far end inward. Together they move γ_1 and γ_2 around the vertical leg.

`f_braid(p, t), f_array_braid(a, p, f0, t)`

```
f_braid(p::Exchange12, t) = (s, n) -> s == 1 ? f_horizontal(p, p.L1 + 1 + n, t) :
                                      s == 2 ? f_horizontal(p, max(p.L1 - n, 0), t) :
                                               f_vertical(p, min(n, p.L2), t)
```

The map onto the 16-site legs of cell 1: arm 1 $n \to i = L_1 + 1 + n$ (the junction is $n = 0$, $i = 15$; $n = 15 \to i = 30 = 2L_1 + 2$); arm 2 $n \to i = \max(L_1 - n, 0)$ (the two outermost sites $n = 14, 15$ share $i = 0$); arm 3 $n \to k = \min(n, L_2)$ (sites $n = 14, 15$ share $k = 14$). `f_array_braid` applies this to cell 1 and `f0` elsewhere.

`struct Exchange23; duration; f_h23, f_v23, f_cell2_23, f_array_23`

```
Base.@kwdef struct Exchange23
    L1::Int = 14; L2::Int = 14; rot_T::Float64 = 100.0; step_T::Float64 = 0.1
end
duration(p::Exchange23) = (p.step_T * (p.L1 + 3p.L2 + 2p.L1 + 3) + 1) * p.rot_T
```

`exchange_23` acts on cell 2 (junction 2, trivial in the initial setup). Duration $(0.1 \cdot 87 + 1)\, T_{\mathrm{rot}}$: 970 for `rot_T = 100` (used in the Hamiltonian check) and 3880 for `rot_T = 400` (used for the X gate, $T_{\mathrm{gate}} = 7760$). `f_h23(p, i, t)` is defined on junction 2's segment $i = 2L_1 + 3 \dots 4L_1 + 4$ $(31 \dots 60)$ and raises an error elsewhere; each half turns topological at one delay and trivial again at a later one, so the segment of cell 2 temporarily connects the two topological segments and γ_2, γ_3 are exchanged through it. `f_v23` has the same form as `f_vertical`. `f_cell2_23` maps the 16-site legs: right arm $i = 3L_1 + 3 + n$ $(45 \dots 60$; junction $i = 45)$; left arm $i = (3L_1 + 2) - (L_1 - 1)\, n/(L_1 + 1)$, a linear map of the 16 sites onto $i = 44 \dots 31$ (non-integer i is allowed since `smoothstep` takes real arguments); vertical $k = \min(n, L_2)$. `f_array_23` applies it to cell 2 and `f0` elsewhere.

`struct Fusion; duration(q::Fusion); f_fuse(q, f0, t)`

```
Base.@kwdef struct Fusion
    L1::Int = 14; Lp::Int = 1; rot_T::Float64 = 100.0; step_T::Float64 = 0.2
end
duration(q::Fusion) = ((q.L1 - q.Lp + 1) * q.step_T + 1) * q.rot_T
```

Partial fusion of pair A on cell 1 (`rot_T0 = 100`, `step_T0 = 0.2`, `Lp = 1`), duration $T_f = 380$. In `f_fuse`: cell 1's vertical leg stays trivial; the sites $n < L_p$ of arm 2 and $n \le L_p$ of arm 1 stay topological (sites $x_c - 1, x_c, x_c + 1$: $2L_p + 1 = 3$ sites); every other site of cell 1 ramps to trivial as $\mathrm{ss}(\tau - 0.2(15 - n))$, the outer end $n = 15$ first. The leg length is hard-coded as `N = 16`. Both halves of the segment thus turn trivial from the outer ends inward.

2.4 Section 3a: time evolution and readout

`struct MovingArrayH; MovingArrayH(a, ft, f0, moving; J, sites); (B::MovingArrayH)(t)`

```
struct MovingArrayH
    a::TArray; ft::Function; moving::Vector{Int}; J::Float64
    H_fixed::MPO; sites::Vector{<:Index}
end
(B::MovingArrayH)(t) = +(B.H_fixed, array_exchange_mpo(B.a, B.sites, B.ft(t), B.moving;
 ↪  J=B.J); cutoff=1e-14)
```

$H(t) = H_{\text{fixed}} + H_{\text{ex}}^{\text{moving}}(t)$. The constructor builds H_{fixed} once: the full MPO at $J = 0$ (chemical potential, hopping, Rashba, pairing) plus the exchange of the cells not in `moving`, computed with `f0`. Calling `B(t)` adds the exchange of the moving cells for texture `ft(t)`. This avoids rebuilding the whole MPO at each time step.

`τz_on(ψ), ΔQ_mps(ψ)`

`τz_on` applies $\tau_z = \text{diag}(1, -1)$ to the Nambu site (the first MPS tensor) and leaves the rest unchanged. `ΔQ_mps(ψ)` $= \text{Re}\langle\psi|\tau_z|\psi\rangle = \sum(|u|^2 - |v|^2)$, the contribution of one mode to ΔQ.

`site_uv(ψ, sites, slist), site_dn(ψ, sites, slist)`

```
"Spin-summed particle |u|² and hole |v|² weights of an MPS at the sites `slist = [(c, s, n),
 ↪  …]` (no dense vector)."
...
        T = head * (ψ[3] * onehot(cl => c))
        for k in 1:L
            T *= ψ[3+k] * onehot(pos[k] => ((n >> (L - k)) & 1) + 1)      # pos[1] = most
             ↪  significant bit
        end
        A = Array(T * (ψ[end] * onehot(arm => s)), ns, ss)
```

Local weights of an MPS at selected sites, without forming the 576-component vector. For each site (c, s, n) the cell, position bits and arm indices are fixed with one-hot vectors; the remaining tensor `head` $= \psi[1]\psi[2]$ (Nambu × spin) gives the 2×2 amplitude array $A_{\tau\sigma}$. Then $|u|^2 = \sum_\sigma |A_{1\sigma}|^2$, $|v|^2 = \sum_\sigma |A_{2\sigma}|^2$. `site_dn` returns $|u|^2 - |v|^2$, the contribution of one mode to δn. *Used in:* the LDOS weights (section 5, $|u|^2 + |v|^2$) and $\delta n(x, t)$ in `evolve_mode_tdvp`.

`mode_leak(z)`

`mode_leak(z) = [1 - sum(abs2, z[:, k]) for k in 1:size(z, 2)]` for the 4×2 overlap matrix $z_{ik} = \langle\gamma_i(0)|\psi_k\rangle$: the leakage ℓ_k of each evolved mode out of the initial zero-mode subspace. This presumes the γ_i orthonormal, which `clean_majoranas` enforces.

`evolve_mode_tdvp(B, ψ0, tsnap; γ0, slist, dt=0.1, log, maxdim=64, cutoff=1e-20, tol=1e-12, krylovdim=30)`

```
function evolve_mode_tdvp(B, ψ0, tsnap; γ0, slist, dt=0.1, log=(args...) -> nothing,
                          maxdim=64, cutoff=1e-20, tol=1e-12, krylovdim=30)
    ...
        n = round(Int, (t1 - t0) / dt); h = (t1 - t0) / n
        ψ = last(TB.evolve_with_tdvp_timedep(t -> B(t0 + t), ψ, n, h; maxdim, cutoff, tol,
         ↪  krylovdim, reverse_step=true))
        record!(i, ψ)
```

Evolves one mode $\psi(t)$ with $i\partial_t\psi = H(t)\psi$ from snapshot to snapshot. Between consecutive snapshot times t_0, t_1 it takes $n = \text{round}((t_1 - t_0)/dt)$ steps of length $h = (t_1 - t_0)/n$ with `TB.evolve_with_tdvp_timedep`: 2-site TDVP in which, on each interval $[t, t+h]$, H is frozen at the midpoint $t + h/2$ (a second-order midpoint rule in time), states are normalized each step, and `reverse_step = true` (the backward local evolution of the TDVP sweep). Settings: `maxdim = 64`, `cutoff = 1e-20` (so the bond dimension is set by the state, not by truncation; the runs reach $\chi = 24$), Krylov exponentiation with `tol = 1e-12`, `krylovdim = 30`.

At every snapshot `record!` stores $z_i = \langle\gamma_i(0)|\psi\rangle$ for the four initial Majoranas, `ΔQ_mps(ψ)`, `site_dn` on `slist` and `maxlinkdim(ψ)`; then `log(i, nt, t, χ, ΔQ)` is called. *Returns* `(; ts, z, dQ, dn, χ, ψT)`; `ψT` is the final MPS.

2.5 Section 3b: zero modes from DMRG, refinement and localization

The zero modes cannot be found by plain DMRG on H (they are in the middle of the spectrum). The notebook minimises $K = H^2 + \eta^2$, whose lowest states are the $|E|$-smallest states of H.

`norm_Hψ(H, ψ)`

$\|H\psi\|$ by exact MPO–MPS application (`alg="naive"`, `truncate=false`); the residual used to judge the zero modes.

`zero_mode_subspace(HB, sites; nzero=2, η=1e-2, seed=1234, nsweeps=20, maxdim=[10,20,50,100,200], cutoff=1e-14, cutoff_K=1e-14, noise=[1e-6,1e-7,1e-8,0], weight=10, eigsolve_krylovdim=20, eigsolve_maxiter=3)`

```
K = TB.build_K(HB, sites, 0.0, η; cutoff_K)        # TB's default cutoff_K = 1e-8 would
 ↪ drop η²
for k in 1:nzero+1
    ψ0 = random_mps(sites; linkdims=10)
    _, ψ = isempty(ψs) ? dmrg(K, ψ0; ...) : dmrg(K, ψs, ψ0; weight, ...)
```

Finds `nzero` states of smallest $|E|$ plus one extra state. *Method:* `TB.build_K(H, sites, ω=0, η)` builds $K = (H - \omega)^2 + \eta^2$ as an MPO (product of MPOs, truncated at `cutoff_K`). The cutoff is set to 10^{-14} because, per the comment, TensorBinding's default 10^{-8} would drop $\eta^2 = 10^{-4}$. DMRG then finds the ground state of K and, one at a time, the next states with a penalty `weight` $\sum_j |\psi_j\rangle\langle\psi_j|$ on the states already found (ITensorMPS excited-state DMRG). The bond dimension grows along `maxdim`; `noise` helps the early sweeps. `eigsolve_krylovdim = 20` is needed: the markdown states that the default stalls at $\|H\psi\| \approx 0.2$.

The last state is removed and used only as a gap check: its $\|H\psi\|$ is the edge of the window (0.268 in section 5). *Returns* `(ψs, [‖Hψ‖ for each], ‖Hψ_gap‖)`.

`refine_subspace(HB, ψs; σ=1e-3, nsweeps=6, cutoff=1e-20, maxdim=200, krylovdim=30)`

```
refine_subspace(HB, ψs; σ=1e-3, ...) =
    [(x = ITensorMPS.linsolve(HB, ψ, ψ, -σ, 1.0; nsweeps, cutoff, maxdim,
                              updater_kwargs=(; krylovdim, verbosity=0)); x / norm(x)) for ψ in
                                ↪ ψs]
```

One inverse-iteration step for each state: solves $(H - \sigma)\, x = \psi$ by DMRG-like sweeps (`linsolve` with $a_0 = -\sigma$, $a_1 = 1$) and normalizes. Components with $|E| \ll$ gap are amplified by $\sim |E_{\text{bulk}}/\sigma|$

relative to bulk admixtures, which the markdown identifies as what ΔQ is sensitive to. The comment records the trade-off for `krylovdim`: the local GMRES solves rarely converge at 30; at 60 they do ($\|H\psi\|$ $2 \times 10^{-5} \to 3 \times 10^{-8}$) but the step is about 2.5 times slower.

`mps_comb(ψs, c; cutoff=1e-14)`

Normalized MPS linear combination $\sum_j c_j \psi_j / \| \cdot \|$, summed pairwise with `cutoff`.

`ritz(HB, ψs)`

Rayleigh–Ritz of H in span(ψ_s): builds the overlap matrix $S_{ij} = \langle \psi_i | \psi_j \rangle$ and $H_{ij} = \langle \psi_i | H | \psi_j \rangle$, solves the generalized Hermitian problem $Hc = ESc$, and returns the energies (ascending) and the Ritz vectors as MPS. This separates the $\pm E$ members, which DMRG on $K = H^2 + \eta^2$ cannot distinguish.

`localize_majoranas(HB, sites, ψs, X)`

```
    F  = eigen(Hermitian((Xm + Xm') / 2), Hermitian((S + S') / 2))
    τx = ITensor([0.0 1.0; 1.0 0.0], ns', ns)
    C(ψ) = (φ = dag(ψ); φ[1] = noprime(τx * φ[1]); φ)
    fix(γ) = exp(im * angle(inner(γ, C(γ))) / 2) * γ
```

Localizes the zero-mode subspace with the position MPO X (`array_position_mpo`): diagonalizes X in the subspace, $X_{ij} c = r\, S_{ij} c$, so each eigenvector is the combination with a sharp position; the eigenvalues are the Majorana centres. It defines the particle–hole conjugation $C\psi = \tau_x \psi^*$ (complex conjugate the MPS, τ_x on the Nambu site), and fixes the global phase of each γ so that $\langle \gamma | C\gamma \rangle$ is real and positive. *Returns* `(; γs, centers, Hγ, C)`.

`clean_majoranas(γs, C)`

```
"""
Make each γ exactly self-conjugate, γ ← (γ + Cγ)/‖·‖, then orthonormalize with the real Löwdin
 ↪  coefficients S^(-1/2)
(⟨γᵢ|γⱼ⟩ is real for self-conjugate γ, so real combinations keep Cγ = γ). The leakage of
 ↪  section 6 assumes both.
"""
```

Projects each γ onto the self-conjugate subspace, then Löwdin-orthonormalizes with $W = S^{-1/2}$ (S real symmetric), so that the γ_i are exactly Majorana and orthonormal. Result in section 5: $\max \|C\gamma - \gamma\| = 1.3 \times 10^{-15}$, $\max |\langle \gamma_i | \gamma_j \rangle - \delta_{ij}| = 1.8 \times 10^{-9}$.

`pair_modes(HB, γs)`

Forms $\psi_A = (\gamma_1 + i\gamma_2)/\sqrt{2}$ and $\psi_B = (\gamma_3 + i\gamma_4)/\sqrt{2}$. If $\langle \psi | H | \psi \rangle < 0$ it flips the sign of γ_2 (or γ_4); for self-conjugate γ this replaces ψ by its particle–hole partner $C\psi = (\gamma_1 - i\gamma_2)/\sqrt{2}$, which has the opposite energy. Both returned modes therefore have $\langle \psi | H | \psi \rangle \geq 0$. *Returns* the (possibly sign-flipped) `γs` and `[ψ_A, ψ_B]`. The `γs` are sorted by their position (from `localize_majoranas`), so pairs (1,2) and (3,4) are the two ends of cell 1's and cell 3's segments.

2.6 Section 4: check of the MPO and export of `tb_reference/`

The cell builds `a = TArray(L=4, ncell=3)`, `HB, sites = msh_tjunction_array(a, f0)` and compares with `array_reference`; it also checks particle–hole symmetry with $C_d = \sigma_x \otimes \mathbb{1}$. Output:

```
D = 576, χ(H) = 18, ‖H_MPO − H_dense‖ = 3.5e-12, PH ‖C H* C + H‖ = 0.0e+00
exchange_12  t = 1020: ‖H_MPO(t) − H_dense(t)‖ = 4.3e-10, χ = 22
exchange_23  t =  485: ‖H_MPO(t) − H_dense(t)‖ = 2.0e-09, χ = 21
fusion       t =  300: ‖H_MPO(t) − H_dense(t)‖ = 1.9e-11, χ = 21
```

The protocol points are mid-exchange ($t = 1020 = T_{12}/2$; $t = 485 = T_{23}/2$ at `rot_T = 100`) and late in the fusion ($t = 300$), each via `MovingArrayH`.

`export_tb_reference(a, H_at; out=joinpath(ROOT, "tb_reference"))`

Writes `coords.txt` (integer x y of each site, order cell, n, arm) and the dense matrices `initial.bin`, `z_t1020.bin`, `x100_t485.bin`, `fuse_t300.bin` (raw `ComplexF64`, Julia column-major). Read by section 4 of `tjunctions_python_ed.ipynb`.

2.7 Section 5: subgap LDOS and MZMs from DMRG

`gauss(x); constants ηg, ωs`

$g_\eta(x) = e^{-x^2/2\eta_g^2}/(\sqrt{2\pi}\eta_g)$ with $\eta_g = 0.01$; frequency grid `ωs` = 401 points on $[-0.2, 0.2]$.

Steps of the section (top-level code).

1. `zero_mode_subspace(HB, sites; nzero=6)`: the six smallest-$|E|$ states (four MZMs and the ± 0.127 bound state of the trivial junction 2) plus the gap-check state; 41 s.
2. `ritz`: energies $[-0.127, -1.12\times10^{-9}, -1.08\times10^{-9}, 1.03\times10^{-9}, 1.13\times10^{-9}, 0.127]$, window edge 0.268.
3. LDOS $A(\mathbf{r},\omega) = \sum_k w_k(\mathbf{r})\, g_\eta(\omega - E_k)$ with $w_k = |u|^2 + |v|^2$ from `site_uv`: total (particle + hole, spin-summed). The window $|\omega| \le 0.2$ lies inside the edge 0.268, so these six states are all that contribute. Check: $\sum_{\mathbf{r}} \int A\, d\omega = 6.0000$; max $A = 17.34$. Written to `msh_array_ldos_tn.csv` (columns `x,y,omega,A`).
4. MZMs: the four Ritz states of smallest $|E|$ → `refine_subspace` ($\sigma = 10^{-3}$) → `localize_majoranas` with `array_position_mpo` → `clean_majoranas` → `pair_modes`, giving `γ0` and `ψ0`; 279 s. Centres $-46.5, -17.5, 16.5, 45.5$; $\langle\psi|H|\psi\rangle = 1.09\times10^{-9}, 1.16\times10^{-9}$; ΔQ of the two modes 2.4×10^{-6} and -2.5×10^{-7}.

`arrows!(p, x, y, u, v; ℓ=0.85, head=0.38, hw=0.42, color, lw, label)`

Plot helper: draws arrows of length ℓ centred on (x, y) along (u, v) in data coordinates (a line for the shaft, a filled triangle for the head, NaN-separated so one series draws all arrows). Used for the in-plane moments in Fig. 1.

2.8 Section 6: gates followed by partial fusion with TDVP

Three runs, each one evolution of ψ_A and ψ_B from `ψ0`:

Run	Gate	Moving cells	T_1	T_{gate}	T_{end}
Z	`exchange_12` twice, then fusion	[1]	2040	4080	4460
X	`exchange_23` (`rot_T = 400`) twice, then fusion	[1, 2]	3880	7760	8140
F	fusion only	[1]	0	0	380

Snapshots every 5 time units during the gate and every 1 during the fusion. For X both cells move: cell 2 during the gate and cell 1 during the fusion.

Horizontal chain. `hsites` lists the 96 sites of $y = 0$ from left to right (per cell: arm 2 $n = 15 \dots 0$, then arm 1 $n = 0 \dots 15$); `xh` their x coordinates. δn is recorded on these sites.

`protocol(kind)`

For `"Z"`, `"X"` or `"F"` returns `(; B, T1, Tg, tsnap)`: a `MovingArrayH` with texture `t < Tg ? fg(t < T1 ? t : t - T1) : f_fuse(q, f0, t - Tg)` (the second exchange repeats the first), the duration of one exchange, the gate duration $T_g = 2T_1$, and the snapshot times `0:5:Tg` followed by `Tg+1:1:Tg+Tf`.

`tdvp_log(key, msg; append=true), stamp(), hms(s); LOG_FILE, STATUS_FILE`

Thread-safe progress reporting (a `ReentrantLock`). `tdvp_log` appends a time-stamped line to `tdvp_progress.log` (if `append`), stores the message as the latest status of evolution `key` (e.g. `"ZA"`), and rewrites `tdvp_status.txt` with one line per evolution. `stamp()` gives the date and time, `hms(s)` formats seconds as h:mm:ss.

`run_mode(kind, k, P)`

Runs `evolve_mode_tdvp(P.B, ψ0[k], P.tsnap; γ0, slist=hsites, log=logf)` for mode k (A or B) of run `kind`. The inner `logf` writes to the status file at every snapshot and to the log at every 1% of T_{end}, with t, χ, the mode's ΔQ, elapsed time and an estimate of the remaining time (elapsed$\cdot(1-f)/f$). Errors are logged as `FAILED` and rethrown. Returns the result merged with the wall time `secs`.

Final Hamiltonian. `HT = protocol("F").B(Tf)` is the Hamiltonian at the end of the fusion (the same for every run). `zero_mode_subspace(HT, sites; nzero=4)`, `refine_subspace` and `ritz` give its four lowest states `φF` with energies $[-0.0439, -1.16\times10^{-9}, 1.16\times10^{-9}, 0.0439]$ (253 s). Pair A is split to $\pm E_A$, pair B stays at zero.

`fusion_leak(ψ)`

```
fusion_leak(ψ) = (c = [inner(φ, ψ) for φ in φF]; 1 - real(c' * (SF \ c)))
```

Leakage of an evolved mode out of span(ϕ_F): $1 - c^\dagger S_F^{-1} c$ with $c_i = \langle\phi_i|\psi\rangle$ and the Gram matrix S_F, i.e. the norm of the projection onto a possibly non-orthogonal basis. The adiabatic ΔQ of pair A is `ΔQ_mps(φF[iA])` for the $+E_A$ state `iA = argmax(EF)`: -0.476854.

`cache(kind); threaded execution`

`cache(kind)` is the path `msh_tjunctions_tdvp_{kind}.jls`. Runs whose cache is missing (or all, if `RUN_TDVP`) are started as `@spawn(run_mode(kind, k, P))` tasks, two per run, so up to six evolutions run in parallel, one per Julia thread, with BLAS set to one thread. For each run the results of both modes are serialized: `ts`, `z`, `dQ`, `dn`, `χ`, `leak_fus = fusion_leak(ψT)`, `secs`, `threads`. Only numbers are cached, not the MPS. In the saved notebook the cache existed (the kernel ran on 1 thread); the timings in the cache are from the six-thread run.

`analyze(kind)`

Loads the cache and returns a `Dict` with `:Tg`, `:ts`, `:dQ` (sum of both modes), `:D` (δn, sum of both modes, snapshot × site), `:χ` (max of both), `:secs`, `:threads`; for gate runs `:leak_one` and `:leak` (`mode_leak` at $t = T_1$ and $t = T_g$); and `:fusion` with the final ΔQ, the per-mode ΔQ, the fusion leakage and the time trace $\Delta Q(t - T_g)$ during the fusion.

`write_density(name, r), density_figure(ts, D, Tg, gate; method)`

`write_density` writes `msh_gate_density_tdvp_{name}.csv` (`t,x,dn`, x fastest) and `msh_gate_dQ_tdvp_{name}.csv` (`t,dQ`). `density_figure` draws two panels: $\log_{10}|\delta n(x,t)|$ over the whole protocol (clipped at 10^{-16}, colour range $[-12, 0]$; dashed lines at $T_g/2$ and T_g, dotted at the junctions) and $\delta n(x,t)$ during the fusion on a symmetric colour scale, with $\sum_{\text{chain}} \delta n$ at the end in the title.

Results. Z: leakage after one exchange $(9.72 \times 10^{-7}, 1.65 \times 10^{-11})$, after two $(1.99 \times 10^{-6}, 2.07 \times 10^{-11})$; fusion $\Delta Q = -0.477717$ (A -0.477717, B 1.4×10^{-7}), leak $(3.69 \times 10^{-5}, 1.44 \times 10^{-11})$. Fusion only: $\Delta Q = -0.477372$, leak $(3.78 \times 10^{-5}, 7.0 \times 10^{-12})$. X: leakage after two exchanges $(3.42 \times 10^{-5}, 3.49 \times 10^{-5})$; fusion $\Delta Q = +0.477004$ (A $+0.476113$, B $+8.9 \times 10^{-4}$), leak $(8.08 \times 10^{-5}, 3.33 \times 10^{-5})$. The sum over the horizontal chain at the end is ∓ 0.444 (figure titles); the rest of ΔQ lies on the vertical leg, since ΔQ is summed over all sites while δn is recorded on $y = 0$ only.

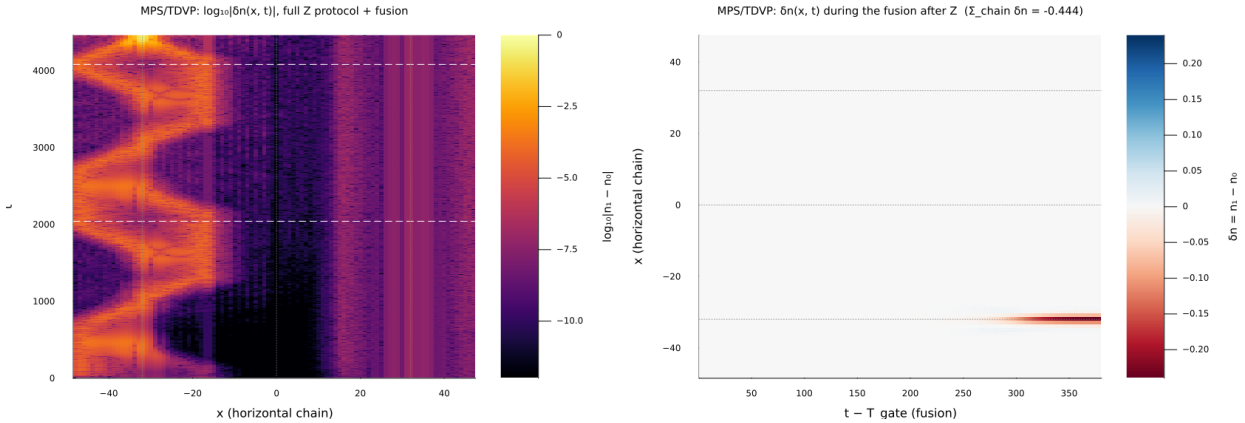


Figure 2: TN, Z run (`density_figure`). Left: $\log_{10}|\delta n(x,t)|$ on the horizontal chain over two `exchange_12` and the fusion (dashed: $T_1 = 2040$, $T_g = 4080$). The weight of ψ_A moves around junction 1 ($x = -32$) while cell 3 ($x > 16$) is static. Right: δn during the fusion; the charge appears at the three topological sites around junction 1.

2.9 Section 7: ΔQ during the fusion and bond dimension

Top-level plotting code (no new functions). Left panel: $\Delta Q(t - T_g)$ for Z, X and F; right: $\max \chi(\psi_A, \psi_B)$ against t/T_{end}, with the start of the fusion marked. The bond dimension saturates at 24 early in every run, well below `maxdim = 64`.

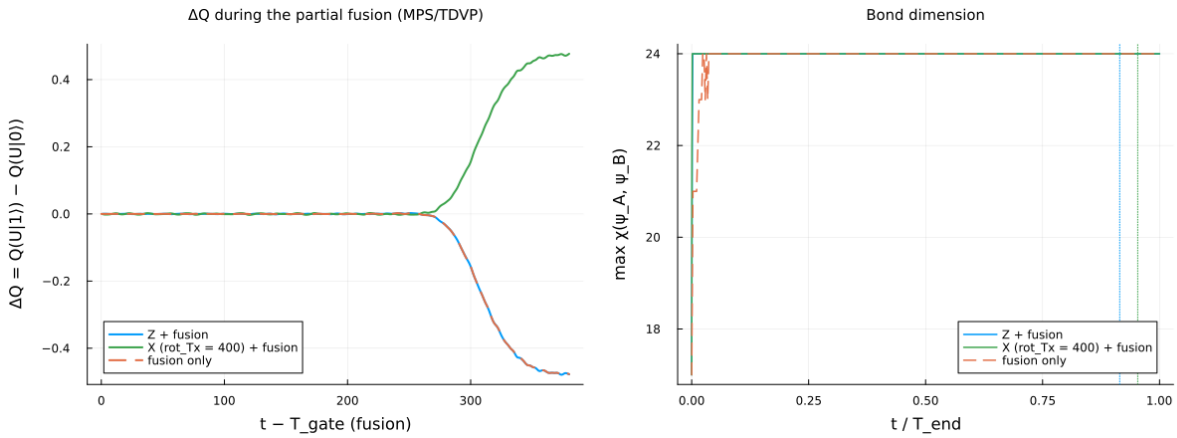


Figure 3: TN, section 7. Left: ΔQ during the partial fusion; Z and no gate give -0.477, X gives $+0.477$. The charge develops for $t - T_g \gtrsim 260$, when the last ramped sites of cell 1 (arm $n = 2$, delay $0.2 \cdot 13$, i.e. $t - T_g = 260$) turn trivial. Right: largest bond dimension of the two modes, saturating at $\chi = 24$.

2.10 Section 8: summary

Top-level code writes 12 `key,value` rows to `msh_tjunctions_tn_gates.csv`: `Z_leak_A/B`, `X_leak_A/B` (after two exchanges), `Z/X/F_fus_dQ`, `Z/X/F_fus_leak_A`, `fus_dQad_A` (-0.476854) and `fus_E_A` (0.0438539). The values appear in Table 1.

2.11 Section 9: cost of the tensor-network route

Measured wall times (from the cache for TDVP; each evolution on one thread, six at once):

Step	Wall time	Steps	Per step
DMRG, six subgap states	0:00:41		
refinement + localization	0:04:39		
DMRG, final Hamiltonian	0:04:13		
TDVP Z, mode A / B	1:17:48 / 1:06:57	44 600	104.7 / 90.1 ms
TDVP X, mode A / B	2:18:53 / 2:16:30	81 400	102.4 / 100.6 ms
TDVP F, mode A / B	0:08:06 / 0:06:32	3 800	127.9 / 103.3 ms

`bench_tdvp(L; χ=24, nsteps=10)`

> `"Time per TDVP step (dt = 0.1, maxdim = 64) for arms of 2^L sites, initial Hamiltonian, random ↪ MPS of bond dimension χ."`

Builds the array with arms of 2^L sites, a `MovingArrayH` with the static texture `f0`, and a random complex MPS of bond dimension 24 (the value reached in section 6). One step is run first to exclude compilation, then `nsteps` are timed with the same TDVP settings as section 6. *Returns* `(; L, nsite, per_step, mps_size, dense_size)`, where `mps_size` is the number of complex entries of the MPS and `dense_size` $= 4\, n_{\text{site}}$.

L	sites	TDVP step	MPS entries	dense entries
4	144	93.3 ms	4845	576
5	288	115.4 ms	5997	1152
6	576	247.3 ms	7149	2304

Doubling the number of sites adds one qubit to the MPS, so its size grows by a constant (≈ 1150 entries) per doubling. All times and sizes go to `msh_tjunctions_tn_timing.csv`.

3 Part II — Python/kwant notebook (`tjunctions_python_ed.ipynb`)

The same geometry, texture and protocols, with exact diagonalization and an exact evolution of the full BdG basis. The gate analysis follows the construction of the user's CZ, QFT and period-finding notebooks: `individual_MZMs`, `fix_unitary`, logical-state `masks` and the Gauss–Legendre `step`.

Where the code lives. The model (sections 1–3), the zero-mode construction of section 6 and the sparse Gauss–Legendre step are defined in `tjunctions_model.py`, which this notebook and `logical_gates_python_ed.ipynb` import (`from tjunctions_model import *`); the notebook keeps the checks, the readout, the runs and the figures. The functions are documented below in the order the notebook uses them, wherever they are defined.

3.1 Setup (cell 1)

Cell 1 sets `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` to 1 before NumPy is imported (`tjunctions_model.py` does the same if it is imported first; the comment: for tiny matrices threaded BLAS only adds contention; it also makes the timings single-thread). `DATA_DIR = "tjunctions_python_ed_data"` receives figures and `.npz` files. The module defines the Pauli matrices `tau_0, tau_x, tau_y, tau_z` and the Kronecker products `txs0` $= \tau_x\sigma_0$, `t0sx`, `t0sy`, `t0sz`, `tzsx`, `tzsy`, `tzs0`, `tysy`, and `tysx`, `tys0` for the unused spinor branches of `PHconj` (first factor Nambu, second spin).

3.2 Section 1: geometry

Constants: `NLEG = 16` (sites per arm, Julia `TArray` with $L = 4$), `NCELL = 3`, `SPACING = 32`, `ARM_ANG`, `ARM_ORIG`, `ARM_DIR` $= ((1,0),(-1,0),(0,1))$.

`center_x(c), array_pos(c, s, n), label`

As in Julia: $x_c = (c-2)\cdot 32$ and $\mathbf{r}(c,s,n) = (x_c, 0) + \mathbf{o}_s + n\hat{e}_s$ (integers). `label` is the dictionary $(x,y) \mapsto (c,s,n)$ over all 144 sites, used to evaluate textures at kwant sites.

`build_system_3T()`

```
def build_system_3T():
    def chain(site, mu, Delta, J, S):
        Sx, Sy, Sz = S[site.tag]
        JS = J * (Sx * t0sx + Sy * t0sy + Sz * t0sz)
        return -mu * tzs0 + Delta * txs0 + JS
    def hop_x(sitei, sitej, t, alpha):
        return -t * tzs0 + 1j * alpha * tzsy
    def hop_y(sitei, sitej, t, alpha):
        return -t * tzs0 - 1j * alpha * tzsx
```

Finalized kwant system on a square lattice with four orbitals per site, basis $(c_\uparrow, c_\downarrow, c_\downarrow^\dagger, -c_\uparrow^\dagger)$. Every site in `label` gets the on-site function `chain`: $-\mu\tau_z + \Delta\tau_x + J\,\tau_0\,\mathbf{S}\cdot\sigma$, with the moment looked up in the parameter `S` (a dict keyed by site tag). `HoppingKind` adds x and y bonds between every pair of existing neighbouring sites, so the junction and inter-cell bonds appear automatically. Output: 144 sites, $x = -48 \ldots 47$, legs at $x = -32, 0, 32$.

3.3 Section 2: spin texture and protocols

Line-by-line ports of the Julia code (Part I, section 2): `smoothstep`, `f0`, classes `Exchange12`, `Exchange23`, `Fusion` with a `duration()` method, `f_horizontal_12`, `f_vertical_12`, `f_braid_12`,

`f_array_12` (Julia `f_array_braid`), `f_h23`, `f_v23`, `f_array_23` (which contains the cell-2 map of Julia's `f_cell2_23`), and `f_fuse(q, t)` (uses `f0` from the global scope and `NLEG` instead of the hard-coded 16). The leg maps are identical, including the clamping of the outer sites ($\max(L_1 - n, 0)$, $\min(n, L_2)$) and the linear map of cell 2's left arm.

`spins(f)`

Returns the dict `S[(x, y)]` $= (\cos\phi \sin\theta, \sin\phi \sin\theta, \cos\theta)$ with $\theta = (-1)^{x+y}(\pi/2)f(c, s, n)$ and $\phi = \theta_{\text{arm}} + \pi/2$ for all sites; the kwant parameter `S`.

3.4 Section 3: Hamiltonian

```
params = dict(t=1.0, mu=-7.0, alpha=0.9, Delta=2.4, J=5.8).
```

`ham_kwant(f, **kw)`

Dense $4N \times 4N$ matrix `sys.hamiltonian_submatrix` for texture f, with `params` overridden by `kw`. Used for the reference checks; `H_noJ = ham_kwant(f0, J=0.0)` is the texture-independent part.

`exchange_matrix(f, J), ham(f)`

`exchange_matrix` fills the 4×4 diagonal blocks $J(S_x\tau_0\sigma_x + S_y\tau_0\sigma_y + S_z\tau_0\sigma_z)$ in kwant's site order (`tags`). `ham(f) = H_noJ + exchange_matrix(f)`, i.e. $H(t) = H_{J=0} + J\sum \tau_0\, \mathbf{S}\cdot\sigma$. The markdown states that this is identical to `ham_kwant` and much faster; the check prints $\|H_{\text{fast}} - H_{\text{kwant}}\| = 0$, Hermiticity error 0 and particle–hole error $\|CH^*C + H\| = 0$ with $C = \mathbb{1} \otimes \tau_y\sigma_y$.

`Cop(v)`

Particle–hole conjugation in the $(u_\uparrow, u_\downarrow, v_\downarrow, -v_\uparrow)$ basis: $Cv = (\mathbb{1}_N \otimes \tau_y\sigma_y)\, v^*$, with $CH^*C = -H$. Used to check that the columns of `U_fixed` are particle–hole partners.

3.5 Section 4: check against the Julia/TensorBinding array

Reads `tb_reference/coords.txt` (raises `FileNotFoundError` with instructions if the TN notebook has not been run) and builds a $4N \times 4N$ matrix `P` that maps kwant's site-major vector to Julia's nambu ⊗ spin ⊗ site order: for Julia site j at kwant index k, `P[b*N + j, 4*k + a] = Wsite[b, a]` with

$$W_{\text{site}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad (u_\uparrow, u_\downarrow, v_\uparrow, v_\downarrow)_{\text{Julia}} = (a, b, -d, c).$$

`to_tb_basis(H)`

PHP^T: the kwant matrix in Julia's basis. Each `.bin` file is read as `complex128`, reshaped in Fortran order. Results: `initial` 3.5×10^{-12}, `z_t1020` 4.3×10^{-10}, `x100_t485` 2.0×10^{-9}, `fuse_t300` 1.9×10^{-11}, the same numbers as the MPO-vs-dense check of the TN notebook.

3.6 Section 5: subgap LDOS

Top-level code: `la.eigh(H0)` for all eigenpairs; `w_site` sums $|U|^2$ over the four orbitals of each site (so it includes particle and hole); `g_eta` is the Gaussian with $\eta = 0.01$; $A = w_{\text{site}}\, g_\eta$ on 401 points of $[-0.2, 0.2]$. All eigenstates are included; the markdown notes that those at $|E| \geq 0.268$ contribute less than e^{-23} inside the window ($(0.268 - 0.2)^2/2\eta^2 \approx 23$). Lowest $|E|$ (one per $\pm$ pair): $0, 0, 0.127, 0.2683$; $\max_{\mathbf{r}} A(\mathbf{r}, 0) = 15.21$. Saved to `array_ldos_kwant.npz` (`x, y, omega, A`).

`ldos_figure(Amat, name, out, vmax)`

Plots $A(\mathbf{r}, \omega = 0)$ on the lattice grid (NaN outside the sites), with the in-plane moments as `quiver` arrows and the $\hat{z}$ moments as dots; saves to `out`. The Python counterpart of the Julia LDOS map (Fig. 1).

3.7 Section 6: Majoranas, fermion modes and logical states

Copied helper functions

`dag(A, axis1=-2, axis2=-1)`

Hermitian conjugate: swaps the last two axes and conjugates; for a 1D array only conjugates.

`PHconj(psi, ordering="ltslts", spinor="udd-u")`

```
def PHconj(psi, ordering="ltslts", spinor="udd-u"):
    """Return the particle-hole conjugate C psi = (tau_y sigma_y) psi^* for the kwant spinor
    ↪  "udd-u".
    ..."""
```

Particle–hole conjugate of the columns of `psi` ($4N_s \times n$). For the site-major ordering `"ltslts"` and the kwant spinor `"udd-u"` it applies $\tau_y\sigma_y$ to each site's four components of ψ^* (the case used here). Other branches: spinor `"uddu"` ($\tau_y\sigma_x$, `tysx`), `"udud"` ($\tau_y\sigma_0$, `tys0`) and a Nambu-outer ordering `"tsltsl"`. The matrices `tysx` $= \tau_y\sigma_x$ and `tys0` $= \tau_y\sigma_0$ are defined in `tjunctions_model.py`, so every branch runs; only the default is used here.

`takagi(A)`

```
def takagi(A):
    """D, U = takagi(A)
    Diagonalize a complex symmetric matrix as A = U D U^T
    where U is unitary and D is diagonal."""
```

Takagi factorization of a complex symmetric matrix. *Method* (*from the code and its comments*): (i) $B = A^\dagger A$ is Hermitian positive semidefinite; diagonalize $B = VbV^\dagger$. (ii) $C = V^T A V$ is complex symmetric with $C = X + iY$, $[X, Y] = 0$; diagonalize $X + Y$ (`eigh`) to get W, a simultaneous eigenbasis, and set $U = VW$. (iii) $z_i = (U^T A U)_{ii} = r_i e^{i\theta_i}$; multiply column i of U by $e^{-i\theta_i/2}$ (`(r/z)**0.5`) so the diagonal becomes real and non-negative. Returns `r` and `U.conj()`.

Why it works (now in the code comments). $C^\dagger C = V^\dagger A^\dagger A V = \mathrm{diag}(b)$ is real, and with $C = X + iY$ (X, Y real symmetric) $C^\dagger C = X^2 + Y^2 + i[X, Y]$, so $[X, Y] = 0$; one real orthogonal W (the eigenvectors of $X + Y$, barring accidental degeneracy) diagonalizes both, and $U^T A U = W^T C W$ is diagonal. After the phase fix $U^T A U = \mathrm{diag}(r)$, i.e. $A = U^* \mathrm{diag}(r) U^\dagger$: the factor Z of $A = ZDZ^T$ is U^*, hence `U.conj()`.

`individual_MZMs(E, M, tol=1e-3, ordering, spinor)`

```
    mask = np.abs(E) < tol
    psi = M[:, mask]
    C = dag(psi) @ PHconj(psi, ordering=ordering, spinor=spinor)
    c, Z = takagi(C)
    gamma = psi @ Z
```

```
x = np.repeat(xs, 4)   # x coordinate of each component (site-major ordering, see the
↪  markdown)
R = dag(gamma) * x @ gamma
r, O = np.linalg.eigh(R.real)  # Want O ortho to conserve C
mbs = gamma @ O
```

Separates individual Majoranas in the zero-energy subspace. (1) The subspace is $|E| < \texttt{tol} = 10^{-3}$; if it has an odd number of states the closest state outside is added with a warning. (2) $C_{ij} = \langle\psi_i|C\psi_j\rangle$ is the (complex symmetric) matrix of the antiunitary C in the subspace; its Takagi vectors Z give $\gamma = \psi Z$, eigenstates of C (Majoranas): with $C_{\text{mat}} = ZDZ^T$, $C\gamma = (C\psi)Z^* = \psi C_{\text{mat}}Z^* = \psi ZD(Z^TZ^*) = \gamma D$, and $D = \mathbb{1}$ for a subspace closed under C. (3) The position operator $R = \gamma^\dagger x\gamma$ is diagonalized with its real part (R is real for Majoranas because C commutes with x), so O is real orthogonal and the γO stay eigenstates of C; the Majoranas come out localized and sorted by x. Returns `mbs, mask, M`.

Change for this geometry (from the markdown): the original localizes with `x = np.tile(range(N//4), 4)`, the site index. For the site-major ordering `4*site + c` this should be `np.repeat`, and here the site index is not ordered along the chain, so the operator uses the x coordinate of each site, `np.repeat(xs, 4)`. The `ordering` and `spinor` arguments are passed on to `PHconj`; the default spinor is `"udd-u"`, kwant's, as in the call (the copied code had `"uddu"` as default and did not pass the arguments on). Output: 4 states, centres $x = -46.6, -17.5, 16.5, 45.6$.

`fix_unitary(mbs, mask, M)`

```
U = np.array([[1, 1], [1j, -1j]]) / 2**0.5
phi = mbs @ np.kron(np.eye(n // 2), U)
ind = np.arange(n).reshape(n // 2, 2).swapaxes(0, 1).ravel()
phi = phi[:, ind]
...
ind = np.arange(n).reshape(2, n // 2); ind[0] = np.flip(ind[0]); phi = phi[:, ind.ravel()]
M[:, mask] = phi
```

Writes fermions into the zero-energy columns so that the full unitary has the BdG form $M = \left(\begin{smallmatrix} U & V^* \\ V & U^* \end{smallmatrix}\right)$. Each consecutive Majorana pair $(\gamma_{2j-1}, \gamma_{2j})$ becomes $(\gamma_{2j-1} \pm i\gamma_{2j})/\sqrt{2}$ (the columns of $U_2 = \frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & 1 \\ i & -i \end{smallmatrix}\right)$). The first reordering (`[0,2,1,3]` for four Majoranas) groups the "+" combinations before the "−" combinations; the second reverses the "+" group. For four Majoranas the zero-energy columns $2N-2, \dots, 2N+1$ (the four middle eigenvectors of `eigh`) become

$$\left(\tfrac{\gamma_3+i\gamma_4}{\sqrt{2}},\ \tfrac{\gamma_1+i\gamma_2}{\sqrt{2}},\ \tfrac{\gamma_1-i\gamma_2}{\sqrt{2}},\ \tfrac{\gamma_3-i\gamma_4}{\sqrt{2}}\right),$$

so column $2N+k$ and column $2N-1-k$ are particle–hole partners ($C\gamma = \gamma$). The notebook calls columns $2N, 2N+1$ ψ_A, ψ_B and requires them to have positive energy. With the Python Majoranas, ψ_A is therefore $(\gamma_1 - i\gamma_2)/\sqrt{2}$; the TN code writes $(\gamma_1 + i\gamma_2)/\sqrt{2}$ with its own sign of γ_2, which is a sign convention for γ_2 only (both are the positive-energy fermion of pair A). The reason for the second reordering (now in the code comment): in the `eigh` ordering of a BdG matrix, $E_{2N+k} = -E_{2N-1-k}$ and these two columns are C partners, so placing φ_k in column $2N-1-k$ and $C\varphi_k$ in column $2N+k$ keeps the fixed M in BdG form and makes the masks (column $2N+k$ occupied ⇔ column $2N-1-k$ empty) valid for any number of pairs. (The copied code computed the Majorana-to-fermion block twice; the duplicate has been removed.) `mbs2 = mbs[:, [0,1,2,3]]` pairs (γ_1, γ_2) and (γ_3, γ_4). Checks printed: column energies $(-1.158, -1.090, 1.090, 1.158) \times 10^{-9}$ (an `assert` requires the last two to be positive; otherwise the order within a pair must be swapped), $\|U^\dagger U - 1\| = 6.5 \times 10^{-14}$, $\max \|C\psi_{2N+k} - \psi_{2N-1-k}\| = 2.2 \times 10^{-14}$. In `tjunctions_model.py`, `zero_mode_frame(H)` runs `eigh`, `individual_MZMs` and `fix_unitary` (with the pairing `mbs[:, [0, 1, 2, 3]]`) and returns `psi0`, the initial full basis, with the eigenvalues, the Majoranas and the zero-energy mask.

Logical states

`masks = logical_masks()` (`tjunctions_model.py`) is a $4\times4N$ boolean array, one row per state $|n_A n_B\rangle$: $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Columns $2N, 2N+1$ are set to the occupations of ψ_A, ψ_B; all higher columns are empty; the lower half is set by `masks[:, :2N] = np.flip(~masks[:, 2N:], axis=1)`, so a column below $2N$ is occupied exactly when its particle–hole partner is empty. Thus $|00\rangle$ occupies all $2N$ negative-energy columns (including $C\psi_A$, $C\psi_B$), and $|11\rangle$ occupies ψ_A, ψ_B instead of $C\psi_A, C\psi_B$. The logical qubit is $|0\rangle_L = |00\rangle$ (row 0), $|1\rangle_L = |11\rangle$ (row 3). `Z0 = psi0[:, 2N-2:2N+2]` is the initial zero-mode subspace.

Sparse $H(t)$ and the Gauss–Legendre step

`exchange_sparse(f, J), ham_sp(f)`

Sparse (CSC) versions: `exchange_sparse` is a `block_diag` of the 4×4 exchange blocks; `ham_sp(f) = H_noJ_sp + exchange_sparse(f)`.

`step(psi, t, dt, ft)`

```
def step(psi, t, dt, ft):
    M0 = -1j * ham_sp(ft(t + c1*dt))
    M1 = -1j * ham_sp(ft(t + c2*dt))
    A = sp.bmat([[I_sp - 0.25*dt*M0, -a12*dt*M0],
                 [-a21*dt*M1, I_sp - 0.25*dt*M1]], format="csc")
    B = np.vstack([M0 @ psi, M1 @ psi])
    lu = splu(A, permc_spec="MMD_AT_PLUS_A")
    ...
    return psi + 0.5*dt*(K[0] + K[1])
```

One step of the 2-stage Gauss–Legendre implicit Runge–Kutta method (4th order) for $\dot\psi = M(t)\psi$, $M = -iH(t)$, applied to all $4N$ columns at once. With nodes $c_{1,2} = \frac{1}{2} \mp \frac{\sqrt{3}}{6}$ and Butcher coefficients $a_{11} = a_{22} = \frac{1}{4}$, $a_{12} = \frac{1}{4} - \frac{\sqrt{3}}{6}$, $a_{21} = \frac{1}{4} + \frac{\sqrt{3}}{6}$, $b_1 = b_2 = \frac{1}{2}$, and $M_i = M(t + c_i\,dt)$ (H at the two Gauss points), the stages solve the linear system

$$\begin{pmatrix} 1 - a_{11}\,dt\,M_0 & -a_{12}\,dt\,M_0 \\ -a_{21}\,dt\,M_1 & 1 - a_{22}\,dt\,M_1 \end{pmatrix} \begin{pmatrix} K_0 \\ K_1 \end{pmatrix} = \begin{pmatrix} M_0\psi \\ M_1\psi \end{pmatrix}, \qquad \psi(t+dt) = \psi + \tfrac{dt}{2}(K_0 + K_1).$$

The $2D \times 2D$ sparse matrix is factorized once per step by `splu` (permutation `MMD_AT_PLUS_A`), and the D right-hand sides are solved in `n_threads` chunks on a thread pool; `n_threads = 1`, so the timings of section 11 are single-thread (the comment notes that more threads are faster). This is the sparse-LU form of the period-finding notebook. Time step `DT = 0.25`.

Readout

`hz` indexes the horizontal chain ($y = 0$) sorted by x; `tauz` $= (1, 1, -1, -1)$ repeated over sites.

`electron_density(W)`

```
"""Electron number per site of the state with occupied columns W: electron part of
↪  diag(G), G = W W†."""
return (np.abs(W) ** 2).sum(axis=1).reshape(N, 4)[:, :2].sum(axis=1)
```

$n(\mathbf{r}) = \sum_\sigma G_{\mathbf{r}\sigma,\mathbf{r}\sigma}$ with $G = WW^\dagger$ restricted to the electron components $(u_\uparrow, u_\downarrow)$.

`readout(psi)`

For the current full basis `psi` returns a dict with `dn_h`: $\delta n = n($`psi[:,masks[3]]`$) - n($`psi[:,masks[0]]`$)$ on the horizontal chain; `dQ`: $\sum_{\mathbf{r}} \delta n$ over all sites; `dQ_modes`: $\langle\psi_k|\tau_z|\psi_k\rangle$ for columns $2N$, $2N+1$; `z`: the 4×2 overlaps $Z_0^\dagger \psi_{A,B}$ with the initial zero-mode subspace. Because the two masks differ only in columns $2N-2 \ldots 2N+1$, `dQ` equals the sum of `dQ_modes` (Overview, Logical qubit and readout); the bulk columns cancel exactly.

`evolve(ft, psi, t0, t1, dt_snap, dt=DT)`

Evolves the full basis from t_0 to t_1 with `step`, renormalizing every column after each step, and calls `readout` at t_0, every `dt_snap` and at the last step. Returns `psi, ts, R` (list of readout dicts).

`mode_leak(R, name)`

Leakage at the last snapshot: $\ell_k = 1 - \|z_{:,k}\|^2$ from `R[-1]["z"]`; prints and returns it. Since `Z0` is an orthonormal basis of the same subspace as the four $\gamma_i(0)$, this equals Julia's `mode_leak(z)`. Values of order -10^{-15} are rounding.

`dQ_adiabatic(H)`

Diagonalizes H and sums $\langle\phi|\tau_z|\phi\rangle$ over the positive-energy states among the four lowest $|E|$, skipping states with $|E| < 10^{-11}$ (an exact zero mode contributes 0). Evaluated at every fusion snapshot to draw the adiabatic curve in the cell-19 plot.

End of the fusion. `q = Fusion()`, `Tf = 380`; `HT_end = ham(f_fuse(q, Tf))`; `Z4_end` are the four eigenvectors of smallest $|E|$. `TIMES` collects wall times.

`run(ft, T1, label, dt_gate=5.0, dt_fuse=1.0)`

One full run starting from `psi0`: if `T1 > 0`, two `evolve` calls of duration T_1 (snapshots every 5) with `mode_leak` after each (`leak_one`, `leak`); then the fusion from $T_g = 2T_1$ to $T_g + T_f$ (snapshots every 1). Fusion leakage $1 - \|Z_{4,\mathrm{end}}^\dagger \psi_k\|^2$ (the counterpart of Julia's `fusion_leak`). Returns `out` with `Tg`, the leakages, `out["fusion"]` (final `dQ`, `dQ_modes`, `leak`, `ts`, `dQ_t`, `dQad_t`) and `out["snap"]` = (times, δn on the horizontal chain, ΔQ) at all snapshots. Records the gate and fusion wall times in `TIMES`.

3.8 Sections 7–8: Z and X gates, then the partial fusion

The texture functions are `fz` (two `exchange_12`, then `f_fuse`), `ff` (fusion only) and `fx` (two `exchange_23` with `rot_T = 400`, then fusion). A check prints $\|H(2T_{12}) - H(0)\| = 0$ with $T_{12} = 2040$: the texture returns to the initial one after the gate. Results:

Run	Leakage (ψ_A, ψ_B)	Fusion: ΔQ (ψ_A, ψ_B); leakage
Z, one exchange	$(9.74 \times 10^{-7}, 0)$	
Z, two exchanges	$(1.997 \times 10^{-6}, 0)$	-0.477717 $(-0.477717, 3.8 \times 10^{-8})$; $(3.6963 \times 10^{-5}, 0)$
F (no gate)		-0.477373 $(-0.477373, 3.8 \times 10^{-8})$; $(3.7778 \times 10^{-5}, 0)$
X, one exchange	$(3.4886 \times 10^{-5}, 1.24 \times 10^{-7})$	
X, two exchanges	$(3.4163, 3.4869) \times 10^{-5}$	$+0.477004$ $(+0.476113, 8.9 \times 10^{-4})$; $(8.075, 3.331) \times 10^{-5}$

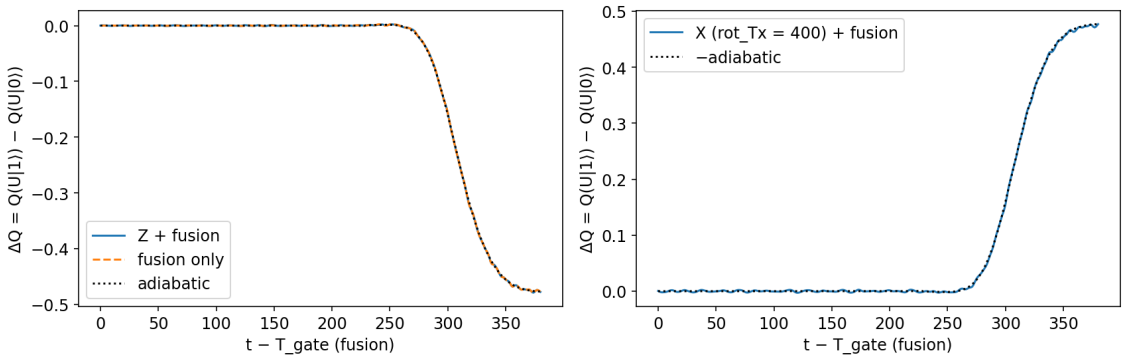


Figure 4: Python, cell 18: ΔQ during the fusion. Left: after Z and with no gate, against the adiabatic value (`dQ_adiabatic`, dotted). Right: after X, against minus the adiabatic value.

3.9 Section 9: charge density $\delta n(x, t)$

`density_figure(ts, x, D, Tg, gate, method, out)`

The matplotlib counterpart of the Julia `density_figure`: $\log_{10}|\delta n|$ over the full protocol (range $[-12, 0]$, dashed at $T_g/2$ and T_g) and δn during the fusion on a symmetric scale; dotted lines at the junctions. Saved to `out`.

The snapshot data of the Z and X runs are saved to `gate_density_kwant_{Z,X400}.npz` (`ts, x, D, Tg, dQ`; `dQ` summed over all sites). Z: $T_g = 4080$, 1197 snapshots, $\Delta Q(T) = -0.477717$; X: $T_g = 7760$, 1933 snapshots, $\Delta Q(T) = +0.477004$.

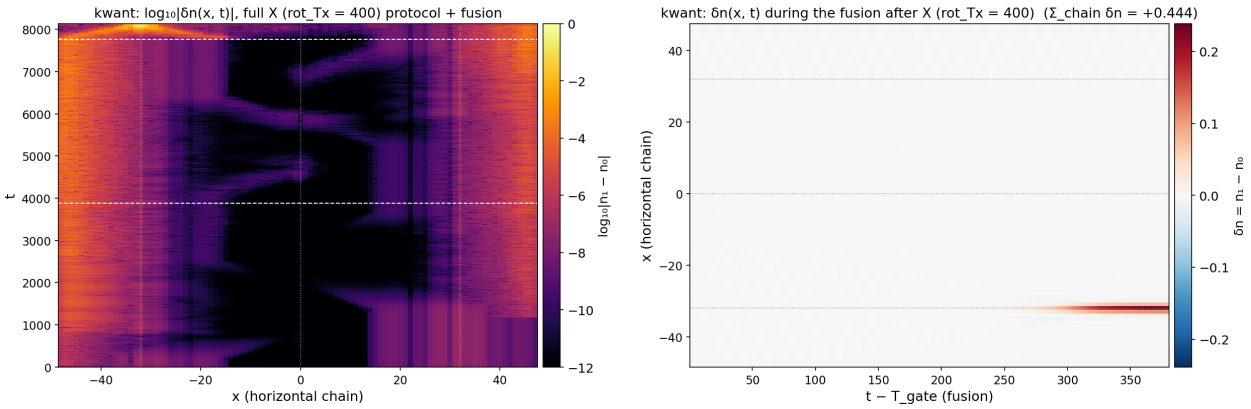


Figure 5: Python, X run (`density_figure`). Left: $\log_{10}|\delta n(x, t)|$ over two `exchange_23` ($T_1 = 3880$) and the fusion. Right: δn during the fusion, with the opposite sign to the Z run ($\sum_{\text{chain}} \delta n = +0.444$).

3.10 Section 10: saving the results

Top-level code: `iA` is the $+E_A$ state among the four lowest $|E|$ of `HT_end`; the dict `gates` holds the same 12 keys as `msh_tjunctions_tn_gates.csv`, with `fus_dQad_A` $= \langle\phi_{iA}|\tau_z|\phi_{iA}\rangle = -0.476854$ and `fus_E_A` $= 0.0438539$. Saved to `gates_kwant.npz`.

3.11 Section 11: cost of the dense route

Measured times (one thread, $dt = 0.25$): Z gate 570.6 s (16 320 steps, 34.96 ms/step), Z fusion 54.4 s (1520 steps), F fusion 54.0 s, X gate 1110.0 s (31 040 steps, 35.76 ms/step), X fusion 54.1 s; one `eigh` of H ($D = 576$) 0.078 s.

`ham_nleg(nleg, f=f0)`

Builds the same array with arms of `nleg` sites (junctions at $(c-2)\cdot 2\,$`nleg`) as a kwant system and returns its sparse BdG matrix and the number of sites. Self-contained copy of the label, texture and kwant builder for arbitrary leg length.

`bench_step(H, dt=DT, nst=3)`

Average time of one full-basis Gauss–Legendre step with a static H, as in `step`: builds M_0, M_1 (as `(H + 0*H).tocsc()`, so a sum of the static H with itself, not the rebuild of $H(t)$ by `ham_sp`), the block matrix, the sparse LU and the solve for D right-hand sides (`psi = I`).

Arm	sites	D	full-basis step	`eigh`
16	144	576	24.6 ms	0.078 s
32	288	1152	157.6 ms	0.544 s
64	576	2304	1095.3 ms	5.649 s

Everything is saved to `timing_kwant.npz` (`time_*`, `steps_*`, `dt`, `eigh`, `bench_*`). The benchmark step at 144 sites (24.6 ms) is faster than the measured step in the runs (about 35 ms). The markdown now explains why: a step of `evolve` also rebuilds the sparse $H(t)$ at both Gauss points (`ham_sp`), goes through the thread-pool solve (which copies B into contiguous chunks and stacks the result, even with one thread) and normalizes ψ, plus a readout every few steps. Timed one at a time on the same machine (under some load, so all times were longer), they cost about 5, 7 and 1 ms per step, which accounts for the gap. The benchmark measures how the step arithmetic scales with D; the extra work is overhead of this implementation, not of the method.

4 Part III — Comparison notebook (`tjunctions_compare.ipynb`)

The notebook only loads the files written by the other two notebooks and compares them, so it runs in seconds. Each comparison row prints max |Julia − Python| against a tolerance that is set by the error of the tensor-network method: the DMRG error for the LDOS and the zero modes, the TDVP error (`maxdim = 64`, $dt = 0.1$) for everything time dependent. The X gate is the run with `rot_Tx = 400`.

4.1 Helper functions

`check(name, a, b, tol)`

```
def check(name, a, b, tol):
    d = float(np.max(np.abs(np.asarray(a) - np.asarray(b))))
    rows.append((name, d, tol))
    print(f"{name:<52s} {d:9.1e}  (tol {tol:.0e})  {'✓' if d <= tol else '✗'}")
```

Maximum absolute difference of two arrays or scalars; appends `(name, d, tol)` to the global list `rows` and prints ✓ if $d \leq$ `tol`, ✗ otherwise.

`read_csv(name), tn_density(name)`

`read_csv` loads a CSV with one header line (`np.loadtxt`). `tn_density` turns a δn CSV (columns `t, x, dn`, x fastest) into the sorted unique times, the sorted unique x and the (t, x) table.

4.2 Section 1: subgap LDOS (row 1)

The Python LDOS (`array_ldos_kwant.npz`, site $\times\omega$) and the Julia CSV (looked up by (x, y, ω) with ω rounded to 6 digits) are compared on all 144 sites and 401 frequencies. Julia uses the six lowest-$|E|$ states from penalized DMRG, Python every eigenstate; inside the window only those six states contribute, so the difference is the DMRG error. Tolerance 10^{-3}; result 3.9×10^{-4}. The two maxima printed in Parts I and II differ (17.34 is the maximum over all ω, 15.21 the maximum at $\omega = 0$).

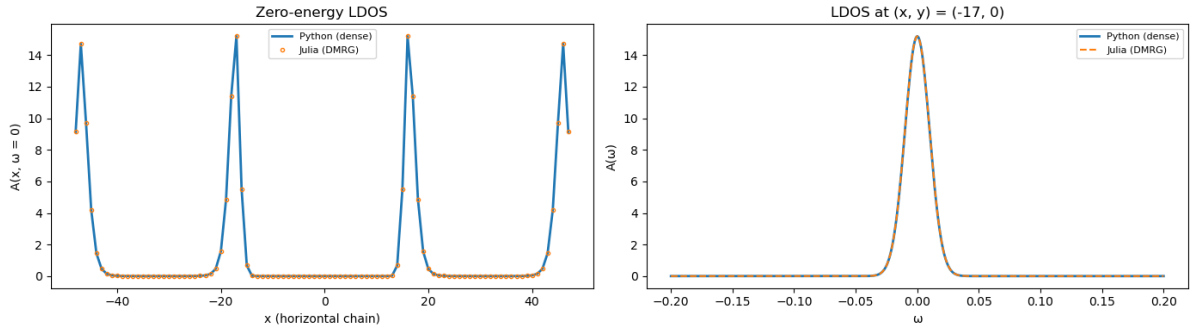


Figure 6: Comparison, section 1: $A(x, \omega = 0)$ along the horizontal chain and $A(\omega)$ at the site of largest zero-energy weight, $(x, y) = (-17, 0)$. Python (dense) lines, Julia (DMRG) markers.

4.3 Section 2: TDVP $\delta n(x, t)$ and $\Delta Q(t)$ (rows 2–5)

For Z and X the Julia CSVs `msh_gate_density_tdvp_{Z,X}.csv`, `msh_gate_dQ_tdvp_{Z,X}.csv` are compared with `gate_density_kwant_{Z,X400}.npz`. An `assert` first checks that the snapshot times and the x grid agree (both codes snapshot every 5 during the gate and every 1 during the fusion). δn is taken on the horizontal chain; $\Delta Q(t)$ is summed over all sites.

Tolerance 10^{-4}, and why. The two time integrators differ. TDVP freezes H at the midpoint of each step (second order, $dt = 0.1$); the Python Gauss–Legendre step is fourth order ($dt = 0.25$) and converged. The markdown states that the midpoint rule at $dt = 0.1$ has a time-step error of a few 10^{-5} in the state, measured against a converged reference, which accumulates over the long X protocol. At the end of each protocol the two agree to $\sim 10^{-7}$ (section 3).

Row	max |Julia − Python|	tol
TDVP $\delta n(x,t)$, Z gate + fusion	7.9×10^{-6}	10^{-4}
TDVP $\Delta Q(t)$, Z gate + fusion	8.0×10^{-6}	10^{-4}
TDVP $\delta n(x,t)$, X gate + fusion	2.3×10^{-5}	10^{-4}
TDVP $\Delta Q(t)$, X gate + fusion	2.6×10^{-5}	10^{-4}

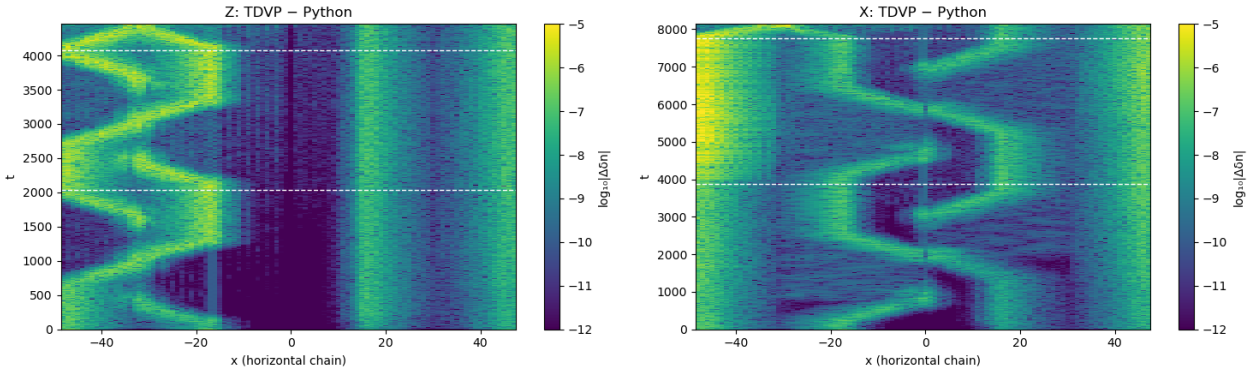


Figure 7: Comparison, section 2: $\log_{10} |\delta n_{\mathrm{TDVP}} - \delta n_{\mathrm{Python}}|$ on the horizontal chain over the full Z and X protocols (dashed: end of the first exchange and of the gate). The differences stay below 10^{-5} and are largest along the paths of the moving Majoranas.

4.4 Section 3: leakage and fusion quantities (rows 6–17)

`msh_tjunctions_tn_gates.csv` and `gates_kwant.npz` are read into dicts with the same 12 keys. Tolerance 10^{-5} for the time-dependent quantities and 10^{-6} for the two static end-of-fusion quantities (`fus_E_A`, `fus_dQad_A`), which come from DMRG in Julia and from diagonalization in Python.

4.5 Results table

Table 1: All 17 comparison rows of `tjunctions_compare.ipynb`, in the printed order. Values from `msh_tjunctions_tn_gates.csv` (Julia) and `gates_kwant.npz` (Python). Result: 17 of 17 within tolerance.

#	Quantity	Julia (TN)	Python	|diff|	tol	
1	LDOS $A(\mathbf{r},\omega)$, DMRG (6 states) vs dense	–	–	3.9×10^{-4}	10^{-3}	✓
2	TDVP $\delta n(x,t)$, Z gate + fusion	–	–	7.9×10^{-6}	10^{-4}	✓
3	TDVP $\Delta Q(t)$, Z gate + fusion	–	–	8.0×10^{-6}	10^{-4}	✓
4	TDVP $\delta n(x,t)$, X gate + fusion	–	–	2.3×10^{-5}	10^{-4}	✓
5	TDVP $\Delta Q(t)$, X gate + fusion	–	–	2.6×10^{-5}	10^{-4}	✓
6	Z: leak ψ_A	1.99193×10^{-6}	1.99727×10^{-6}	5.3×10^{-9}	10^{-5}	✓
7	Z: leak ψ_B	2.07467×10^{-11}	-1.33×10^{-15}	2.1×10^{-11}	10^{-5}	✓
8	X: leak ψ_A	3.41817×10^{-5}	3.41631×10^{-5}	1.9×10^{-8}	10^{-5}	✓
9	X: leak ψ_B	3.48854×10^{-5}	3.48691×10^{-5}	1.6×10^{-8}	10^{-5}	✓
10	fusion ΔQ after Z	-0.477717	-0.477717	1.5×10^{-7}	10^{-5}	✓
11	fusion ΔQ after X	$+0.477004$	$+0.477004$	1.9×10^{-7}	10^{-5}	✓
12	fusion ΔQ, no gate	-0.477372	-0.477373	5.4×10^{-7}	10^{-5}	✓
13	fusion leak ψ_A after Z	3.6929×10^{-5}	3.6963×10^{-5}	3.4×10^{-8}	10^{-5}	✓
14	fusion leak ψ_A after X	8.07556×10^{-5}	8.0751×10^{-5}	4.6×10^{-9}	10^{-5}	✓
15	fusion leak ψ_A, no gate	3.77714×10^{-5}	3.77783×10^{-5}	6.9×10^{-9}	10^{-5}	✓
16	adiabatic ΔQ of pair A (end of fusion)	-0.476854	-0.476854	7.1×10^{-9}	10^{-6}	✓
17	E_A (end of fusion)	0.0438539	0.0438539	4.0×10^{-15}	10^{-6}	✓

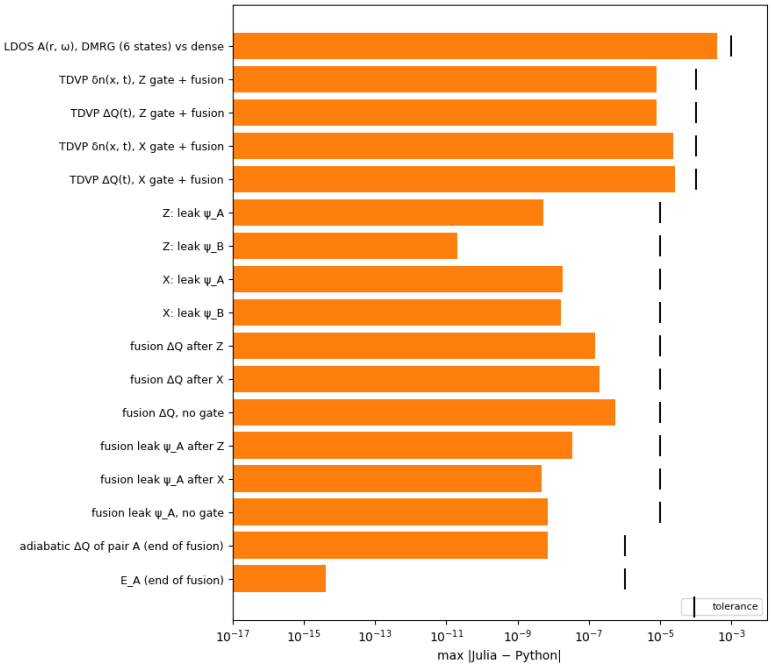


Figure 8: Comparison, section 5: max |Julia − Python| of every row (bars) against its tolerance (black ticks), log scale.

4.6 Section 4: cost and scaling

The two routes evolve different objects. Julia evolves the two zero-mode fermions as MPS (2-site TDVP, $dt = 0.1$, one task per mode); Python evolves the full $4N \times 4N$ BdG basis (sparse Gauss–Legendre, $dt = 0.25$), from which it reads the charge density of the logical states. Both run each evolution on one thread; in Julia the six evolutions ran at the same time on six threads, which the markdown says slows each one a little (shared memory bandwidth).

Measured runs (144 sites, $D = 576$). Julia time = sum of the two TDVP evolutions (they ran concurrently, so the wall-clock time of a run is about half of this); Python time = gate + fusion of the full-basis evolution.

Run	TDVP steps	Julia time	full-basis steps	Python time	ratio
Z gate + fusion	44 600	144.7 min	17 840	10.4 min	13.9×
X (`rot_Tx = 400`) + fusion	81 400	275.4 min	32 560	19.4 min	14.2×
fusion only	3 800	14.6 min	1 520	0.9 min	16.3×
zero modes*		320 s		0.08 s	

* Julia: DMRG of the six subgap states + refinement and localization; Python: one `eigh`.

Scaling. Cost per unit of simulated time (the methods use different steps): Julia 2×`bench_step_L`/0.1 (both modes), Python `bench_step`/0.25. Memory: MPS entries of the two modes, against D^2 entries of the evolved full basis (and $2D$ for two dense vectors, plotted for reference). A straight-line fit of log(cost) against log(sites) gives the local power law.

sites	D	TDVP (s / unit t)	full basis (s / unit t)	`eigh`	MPS entries	full basis D^2
144	576	1.87	0.10	0.08 s	9 690	331 776
288	1152	2.31	0.63	0.54 s	11 994	1 327 104
576	2304	4.95	4.38	5.65 s	14 298	5 308 416

Local power laws, cost $\propto$ sites^p: TDVP $p = 0.70$, full-basis evolution $p = 2.74$. At 576 sites the two costs per unit time are nearly equal (4.95 vs 4.38 s); the dotted extrapolation in the figure is the fitted power law.

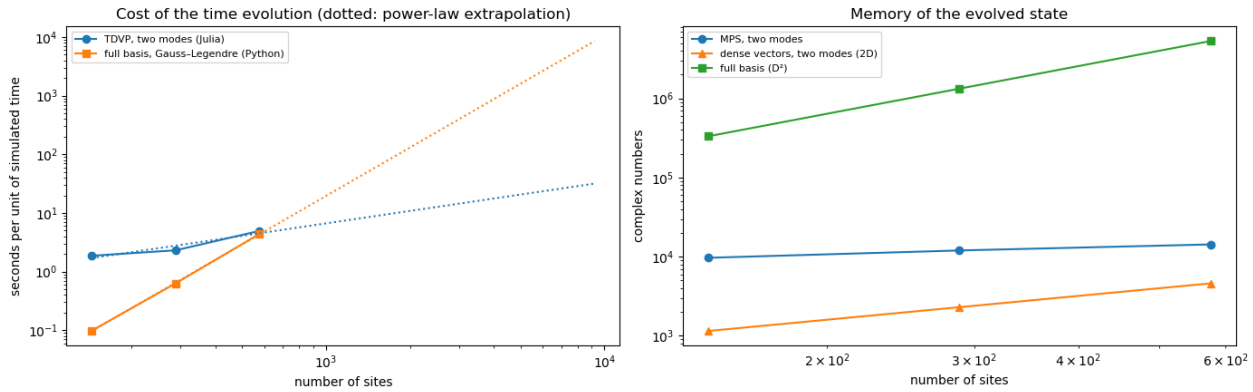


Figure 9: Comparison, section 4. Left: seconds per unit of simulated time for TDVP (two modes) and the full-basis Gauss–Legendre evolution, with power-law extrapolations (dotted). Right: number of complex numbers stored for the evolved state.

4.7 Section 5: summary

Counts the rows within tolerance (17 of 17), lists any failures (none) and draws the bar chart above.